METODOLOXÍAS DE DESENVOLVEMENTO

Metodologías Ágiles: SCRUM

Anxo Pérez & Javier Parapar

anxo.pvila@udc.es

Information Retrieval Lab
Computer Science Department
University of A Coruña

Origen

Origen

“Bajo un enfoque de ‘carrera de relevos’ en el desarrollo de productos ... puede entrar en conflicto con los objetivos de máxima velocidad y flexibilidad. En su lugar, un enfoque holístico o estilo ‘rugby’ - donde un equipo intenta correr como una unidad, pasando la pelota hacia adelante y hacia atrás -pueden servir mejor a los actuales requisitos competitivos”

Takeuchi and Nonaka - Harvard Business Review

Melé

> *scrum (noun): a way of starting play again in which players from each team come together and try to get control of the ball by pushing against each other and using their feet when the ball is thrown in between them*
>
> *Merriam-Webster dictionary*

Melé

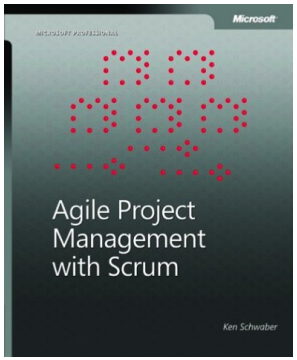


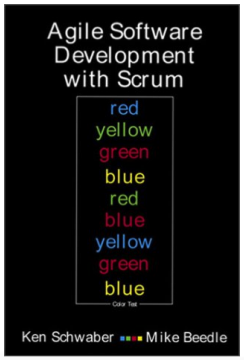
Scrum en 100 palabras

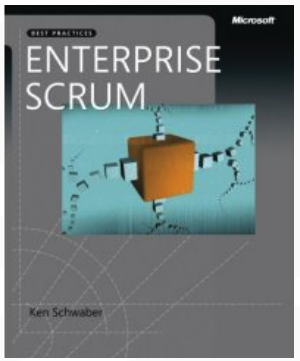
- Scrum es un proceso ágil que nos permite centrarnos en ofrecer **el mayor valor posible** al negocio en el menor tiempo.
- Nos permite **rápida y frecuentemente** inspeccionar software real de trabajo
- El negocio fija las **prioridades**. Los equipos se auto-organizan a fin de determinar la mejor manera de entregar las funcionalidades de prioridad más alta
- Cada dos semanas o un mes, cualquiera puede ver el software real funcionando y decidir si **liberarlo** o seguir **evolucionándolo** en otro ***sprint***

Origen

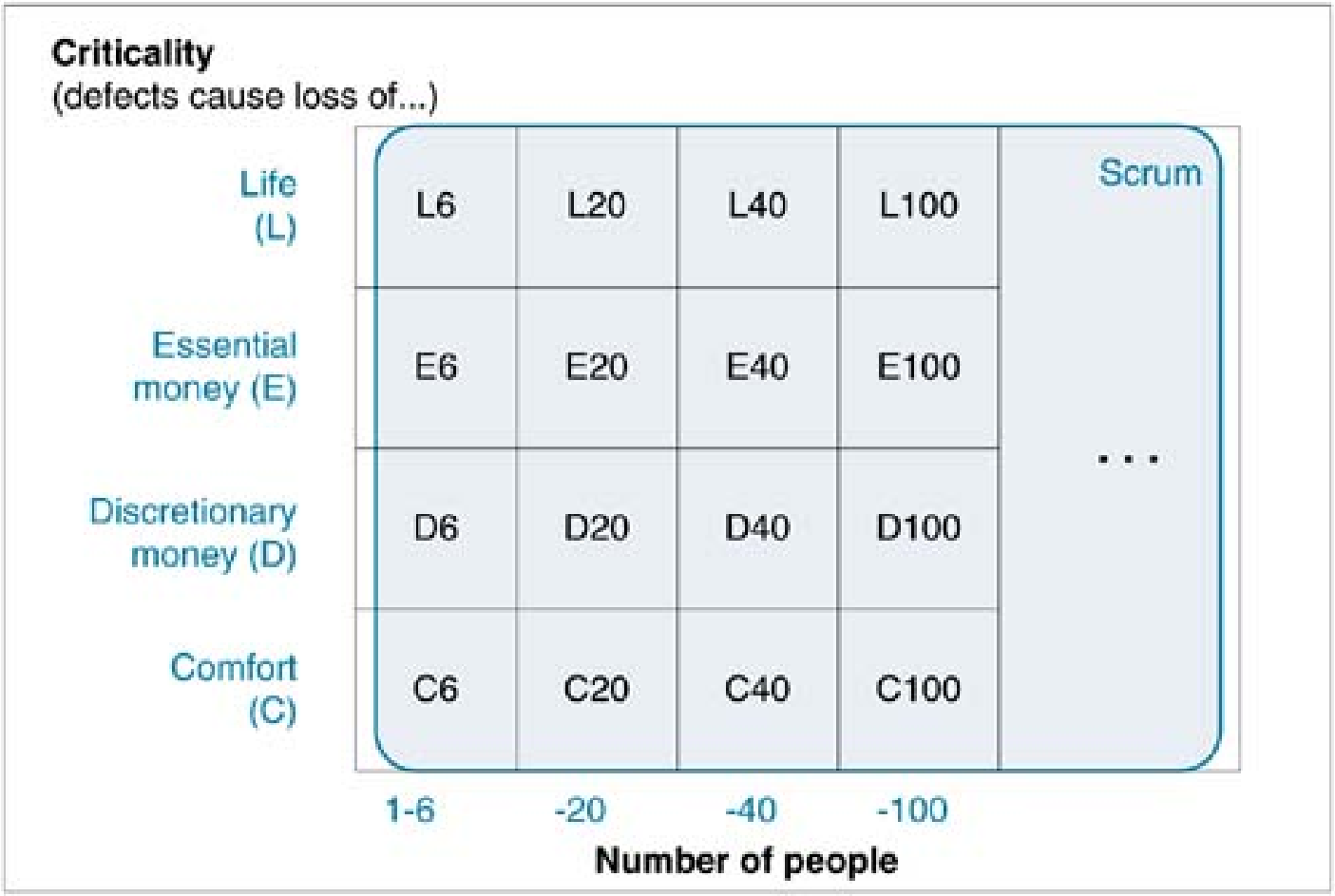
- Jeff Sutherland
  - Scrums iniciales en Easel Corp en 1993
  - IDX (*GE Healthcare*) 500 personas haciendo Scrum
- Ken Schwaber
  - ADM (Advanced Development Methods)
  - Se presenta Scrum en OOPSLA'95 (*Object-Oriented Programming, Systems, Languages & Applications*) con Sutherland en 1993
- Mike Beedle
  - Patrones Scrum en PLOP'04 (*Pattern Languages of Programs Conference*)
- Ken Schwaber y Mike Cohn
  - Fundaron conjuntamente la Scrum Alliance en 2002, inicialmente dentro de la Agile Alliance







¿Para qué proyectos es posible usar Scrum?



Scrum cubre toda la escala Cockburn

¿Quién usa Scrum?

- Microsoft
- Yahoo
- Google
- Electronic Arts
- Banco Santander
- Lockheed Martin
- Philips
- Siemens
- Nokia
- BBVA
- BBC
- Telefónica
- Repsol
- Nielsen Media
- NASA
- Facebook
- Volkswagen
- John Deere
- Lexis Nexis
- Amazon
- Salesforce.com
- Time Warner
- Turner Broadcasting
- Intel

Scrum ha sido usado para...

- Software comercial
- Desarrollos internos
- Desarrollos bajo contrato
- Proyectos presupuesto cerrado
- Aplicaciones Financieras
- Aplicaciones certificadas ISO 9001
- Sistemas Embebidos
- Sistemas con requisitos 7x24
- Joint Strike Fighter (Avión combate)
- Desarrollo de video juegos
- Sistemas críticos de soporte vital (FDA)
- Software de control de satélites
- Sitios web
- Software para tablets y dispositivos móviles
- Aplicaciones de red
- Aplicaciones bajo licencia
- Grandes aplicaciones

Principales características

- Equipos auto-organizados.
- El producto progresa en una serie de sprints de duración semanal/mensual.
- Los requisitos se capturan como elementos de una lista llamada product backlog.
- No existen prácticas de ingeniería prescritas.
- Emplea normas generativas para crear un entorno ágil para la entrega de proyectos.

Roles técnicos

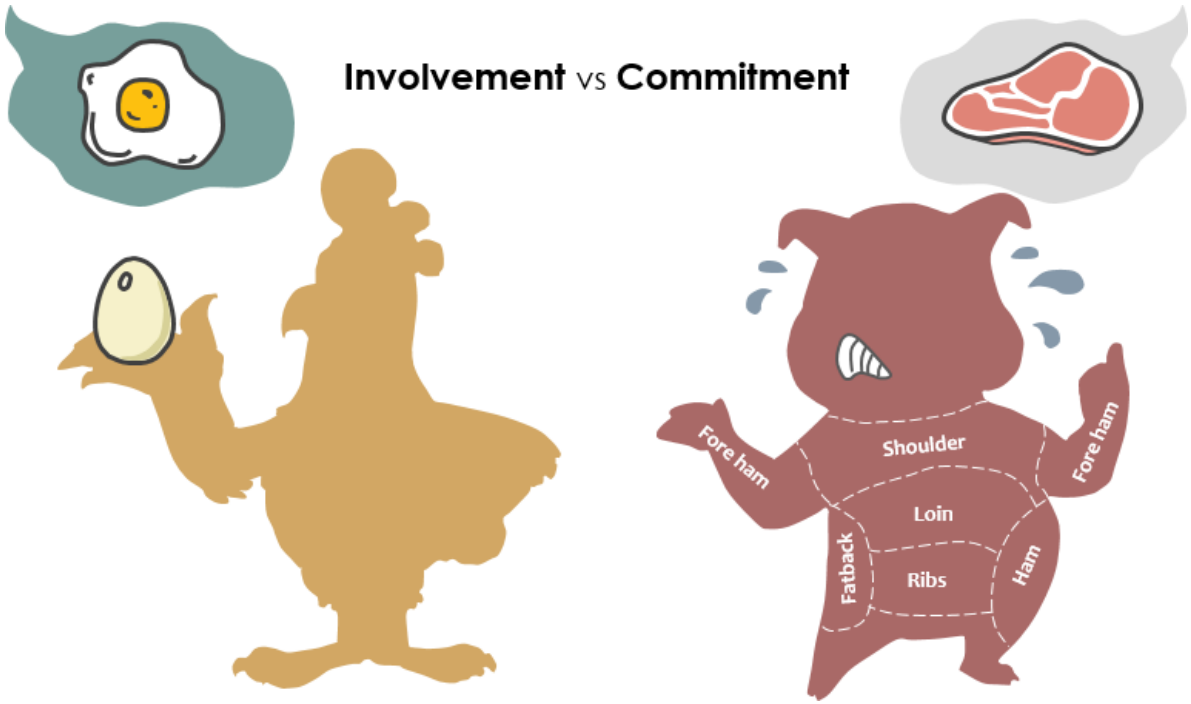
La historia de la gallina y el cerdo

En los primeros materiales de Scrum se utilizaba esta historia para distinguir entre **participar** y **asumir responsabilidad** por el trabajo.

A chicken and a pig are discussing opening a restaurant:

- **Chicken:** "We could serve ham and eggs!"
- **Pig:** "You would provide the eggs, but I would have to give my life to provide the ham."
- Hence the phrase: "You would be involved, but I would be fully committed."

¿Qué representa la historia?



- **Gallina:** Aporta algo al restaurante y puede seguir poniendo huevos. Su contribución no le exige el mismo sacrificio.
- **Cerdo:** Su aportación implica una consecuencia irreversible. El chiste exagera la diferencia entre ambas situaciones.

¿Por qué se contaba en Scrum?

- Los **«cerdos»** representaban al equipo Scrum: Product Owner, Scrum Master y desarrolladores. Cada uno asume responsabilidades concretas sobre el producto y el trabajo.
- Las **«gallinas»** representaban a otros interesados, como usuarios o responsables de negocio, que aportan necesidades y feedback.
- La idea era distinguir entre **aportar una opinión** y **responder por las decisiones y su ejecución**.
- La metáfora tiene límites: los interesados también pueden estar muy comprometidos y verse muy afectados por el resultado.

Metáfora histórica, retirada de la Guía Scrum en 2011. Fuente: revisiones de la Guía Scrum.

Ejemplo: una tienda en línea

- **Usuarios y negocio:** Detectan que muchos compradores abandonan el carrito y proponen simplificar el pago.
- **Product Owner:** Valora esa necesidad frente a otras y decide su orden en el Product Backlog.
- **Desarrolladores:** Planifican cómo construir y probar la mejora. El Scrum Master ayuda a mejorar la eficacia del equipo y a eliminar impedimentos.
- **Colaboración:** El equipo y los interesados revisan el resultado y las posibles mejoras. La aportación de los usuarios sigue siendo esencial.

Idea que conservamos: Responsabilidades claras y colaboración entre todas las partes.

Roles técnicos: Propietario del Producto

- El propietario del producto (**product owner**) es quien toma las decisiones del **cliente**, su responsabilidad es el **valor del producto**
- Para simplificar la **comunicación y toma de decisiones** es necesario que este rol recaiga en una **única persona**
- Si el cliente es una organización puede adoptar la forma de comunicación interna que consideren oportuna, pero en el equipo de desarrollo **sólo se integra una persona** en representación del cliente, y ésta debe tener el **conocimiento** suficiente del producto y las **atribuciones** necesarias para tomar las **decisiones**



Roles técnicos: Propietario del Producto

- Decide en última instancia cómo será el **resultado final**, y participa en el **orden** en el que se van construyendo los sucesivos **incrementos**: qué se pone y qué se quita de la pila del producto, y cuál es la prioridad de las funcionalidades
- Conoce el **plan** del producto, sus **posibilidades** y plan de **inversión**, así como del **retorno** esperado a la inversión realizada, y se responsabiliza sobre **fechas** y funcionalidades de las diferentes **versiones** del mismo

Roles técnicos: Propietario del Producto

- En los desarrollos **internos** para la propia empresa, suele asumir este rol el ***product manager*** o el responsable de marketing
- En desarrollos para clientes **externos**, será el responsable del proceso de adquisición del cliente.
- Según las **circunstancias** del proyecto es posible incluso que **delegue** en el equipo de desarrollo, o en alguien de su confianza, pero la responsabilidad siempre es suya

Roles técnicos: Equipo de desarrollo

- Lo forman el **grupo** de profesionales que realizan el **incremento** de cada **Sprint**.
- Se recomienda que un equipo scrum tenga entre **4 y 8 personas**. Más allá de 8 resulta más difícil mantener la comunicación directa
- En el cómputo del número de miembros del equipo de desarrollo no se consideran ni el **Scrum Master** ni el **Product Owner**
- No se trata de un grupo de trabajo formado por un arquitecto, diseñador o analista, programadores y testers. Es un equipo **multifuncional**, en el que todos los miembros trabajan de forma solidaria con responsabilidad compartida.
- Es posible que algunos miembros sean especialistas en áreas concretas, pero la **responsabilidad** es el incremento de cada sprint y recae sobre el equipo de desarrollo en **conjunto**

Tamaño del equipo de desarrollo

- **Two-Pizza Team (4-8 / 5-9 members)**:
  - Se vaguea menos.
  - Fomenta las interacciones constructivas.
  - Menos tiempo invertido en coordinación.
  - Nadie puede ocultarse en segundo plano.
  - Los equipos pequeños suelen ser más satisfactorios para sus miembros.
  - Se evita la sobre-especialización, todos los miembros tienden a asumir múltiples roles y responsabilidades.

Roles técnicos: Equipo de desarrollo

- Las principales **responsabilidades**, más allá de la autoorganización y uso de tecnologías ágiles, son las que se marcan la diferencia entre **grupo de trabajo** y **equipo**
- Un **grupo de trabajo** es un conjunto de personas que realizan un trabajo, con una asignación específica de **tareas**, **responsabilidades** y siguiendo un **proceso** o pautas de ejecución.
- Por ejemplo, los operarios de una cadena, forman un grupo de trabajo: aunque tienen un jefe común, y trabajan en la misma organización, cada uno responde por su trabajo

Roles técnicos: Equipo de desarrollo

- El **equipo** tiene espíritu de **colaboración**, y un **propósito común**: conseguir el mayor valor posible para la visión del cliente
- Un equipo Scrum responde en su conjunto y trabaja de forma **cohesionada** y **autoorganizada**.
- En un equipo **no** hay un **gestor** para delimitar, asignar y coordinar las tareas, son los propios miembros los que lo realizan

Roles técnicos: Equipo de desarrollo

En un equipo de desarrollo SCRUM

- Todos conocen y comprenden la **visión** del **propietario del producto**
- **Aportan y colaboran** con el propietario del producto en el desarrollo de la pila del producto
- Comparten de forma conjunta el **objetivo** de cada *sprint* y la **responsabilidad** del logro
- Todos los miembros participan en las **decisiones**
- Se **respetan** las opiniones y aportes de todos
- Todos conocen el **modelo** de trabajo con **SCRUM**



Roles técnicos: Scrum Master

- Es el responsable del **cumplimiento de las reglas** de un marco de scrum técnico, asegurando que se entienden en la organización, y se trabaja conforme a ellas
- **Asesora y forma** al propietario del producto y al equipo
- Realiza su trabajo con un modelo de liderazgo servil: al **servicio** y en ayuda del equipo y del propietario del producto
- Proporciona:
  - Asesoría y formación al equipo para trabajar de forma autoorganizada y con responsabilidad de equipo
  - Revisión y validación de la pila del producto (requisitos)
  - Moderación de las reuniones
  - Resolución de impedimentos que en el sprint pueden entorpecer la ejecución de las tareas
  - Gestión de las *dinámicas de grupo* en el equipo

Roles técnicos: Scrum Master

- Proporciona:
  - Configuración, diseño y mejora continua de las prácticas de SCRUM en la organización.
  - Respeto de la organización y los implicados, con las pautas de tiempos y formas de scrum
- Al crecer la fluidez de la organización y evolucionar hacia un marco de SCRUM más **pragmático**, puede **eliminarse** el rol de Scrum Master, cuando estas responsabilidades ya estén institucionalizadas en la organización

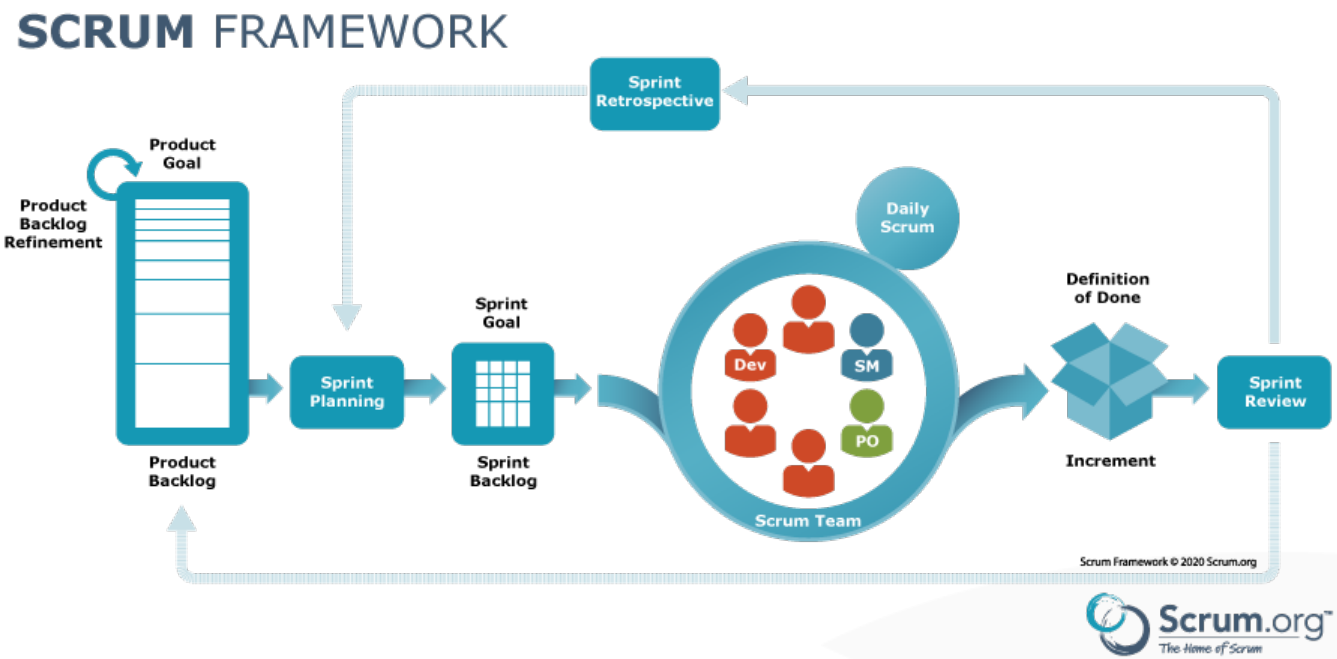
Artefactos

Artefactos

Los principales artefactos del marco de Scrum técnico son:

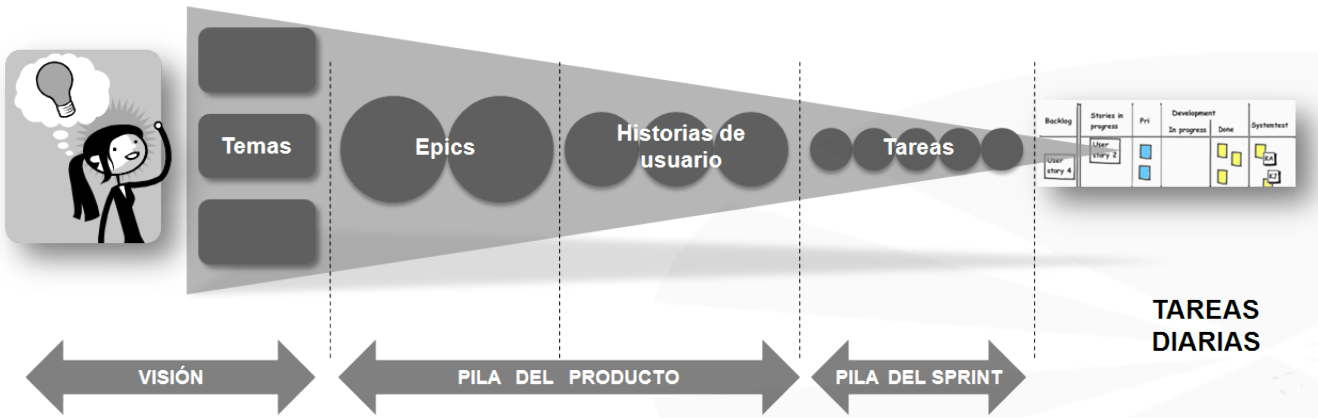
- Pila del producto (**Product Backlog**): Lista de requisitos de usuario priorizada, que, a partir de la visión inicial del producto crece y evoluciona durante el desarrollo
- **Sprint**: nombre que recibe cada iteración de desarrollo. Es el núcleo central que genera el pulso de avance por tiempos prefijados (**Timeboxing**)
- Pila del sprint (**Sprint Backlog**): lista de los trabajos que debe realizar el equipo durante el sprint para generar el incremento previsto
- **Incremento**: producto funcional resultado de cada sprint
- **Definition of Done (Definición de Hecho)**: Es una lista de criterios que define cuándo un trabajo está completamente terminado

Artefactos


SCRUM FRAMEWORK
Sprint Retrospective
Product Goal
Product Backlog Refinement
Product Backlog
Sprint Planning
Sprint Goal
Sprint Backlog
Daily Scrum
Dev
SM
PO
Scrum Team
Definition of Done
Increment
Sprint Review
Scrum Framework © 2020 Scrum.org
Scrum.org
The Home of Scrum

Artefactos: Product Backlog

- **Lista ágil de los requisitos** del **PO** o requisitos del sistema:
  - **Simples**: Los requisitos se expresan de manera concisa y clara. A menudo se usan historias de usuario para capturar las funcionalidades deseadas.
  - **Estimados**: Cada requisito tiene una estimación de esfuerzo (típicamente en puntos de historia o horas).
  - **Priorizados**: Los requisitos están ordenados según la importancia o valor para el product owner.



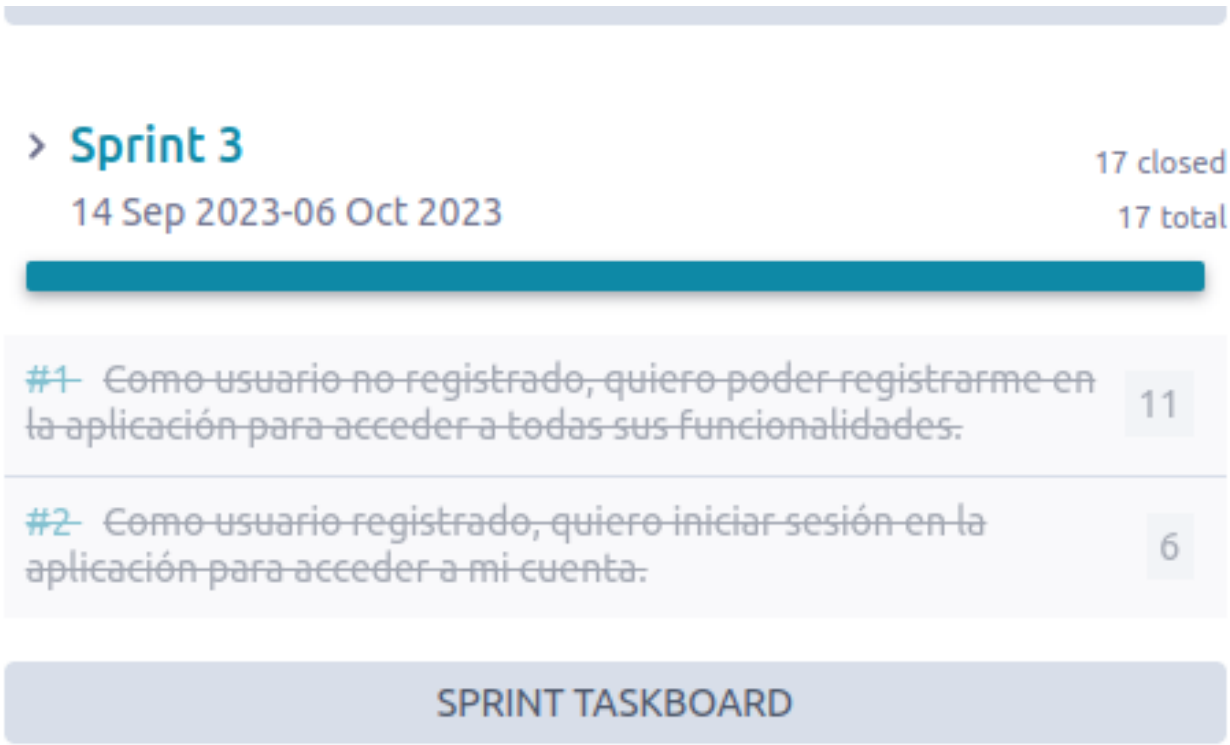
Historias de Usuario

- Una **historia de usuario** es una técnica usada para capturar requisitos desde la perspectiva del usuario final. Es una descripción corta de la funcionalidad deseada, sin entrar en detalles técnicos.
- **Formato:**
  - *"Como [rol del usuario], quiero [acción] para [beneficio]".*
    - "Como administrador, quiero dar de alta nuevos elementos en el catálogo, clasificados por categoría, para que los usuarios puedan seleccionarlos."
    - "Como responsable de un equipo, quiero descargar la actividad de sus miembros en formato CSV para analizar su evolución."

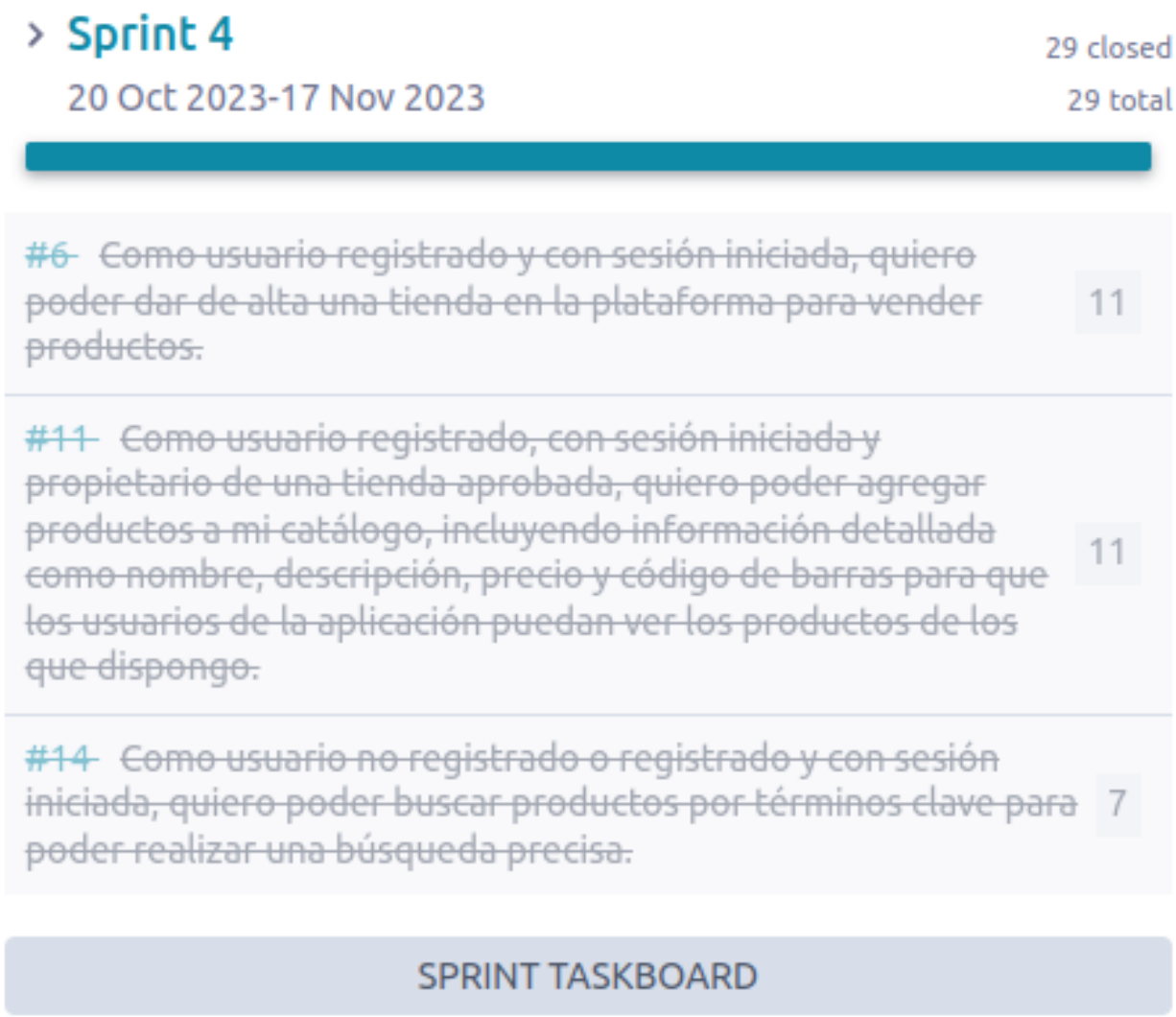
Historias de Usuario

- **Características:**
  - **Concreta**: Una historia debe tener una funcionalidad clara.
  - **Valor**: Debe aportar valor al cliente o al producto.
  - **Negociable**: No es un contrato cerrado, puede cambiar con retroalimentación.
  - **Testable**: Debe ser verificable a través de pruebas claras.

Historias de Usuario



Historias de Usuario



Artefactos: Product Backlog

- La pila del producto es el inventario de **funcionalidades**, **mejoras**, tecnología y **corrección** de **errores** que deben incorporarse al producto a través de los sucesivos sprints
- Representa todo aquello que esperan el cliente, los usuarios, y en general los interesados. **Todo** lo que suponga un **trabajo** que debe realizar el equipo debe estar **reflejado en esta pila**
- **Evolución continua:**
  - No es estática. La pila de producto crece y cambia continuamente en respuesta a nuevos descubrimientos y cambios en el entorno.
- Al comenzar el proyecto incluye los requisitos inicialmente conocidos y mejor entendidos, y conforme avanza el desarrollo, y evoluciona el entorno en el que será usado, se va desarrollando

Artefactos: Product Backlog

- Para comenzar el desarrollo se necesita la visión del **objetivo de negocio** que se quieren conseguir con el proyecto, comprendida y conocida por todo el equipo, y elementos suficientes en la pila para llevar a cabo el primer sprint
- Habitualmente se comienza a elaborar la pila con el resultado de una reunión de ***brainstorming***, o un proceso de exploración tipo *eXtreme Programming* donde colabora todo el equipo partiendo de la **visión del propietario del producto**
- El formato de la visión no es relevante. Según los casos, puede ser una **presentación** informal del responsable del producto, un **informe de requisitos** del departamento de marketing, u otros
- Lo que sí es importante es disponer de una **visión real, comprendida y compartida** por todo el equipo

Artefactos: Product Backlog

- El propietario del producto mantiene la **pila ordenada por la prioridad** de los elementos, siendo los más prioritarios los que confieren mayor valor al producto, o por alguna razón resultan más necesarios, y determinan las actividades de desarrollo inmediatas
- El **detalle** de los requisitos en la pila del producto debe ser proporcional a la **prioridad**: Los elementos de mayor prioridad deben tener mayor nivel de comprensión y detalle que los del resto
- Así el equipo de desarrollo puede descomponer un elemento de prioridad alta en tareas con la precisión suficiente para ser hecho en un sprint
- Los elementos de la pila del producto que pueden ser incorporados a un sprint se denominan **preparados** (*ready*) o **accionables** (*actionable*) y son los que pueden seleccionarse en la reunión de planificación del sprint

Product Backlog ⇒ Preparación

- Se denomina preparación (***grooming***) de la pila del producto a las actividades de **priorización**, **detalle** y **estimación** de los elementos que la componen
- Es un proceso que realizan de forma **puntual**, en cualquier momento, **continua** y **colaborativa** el propietario del producto y el equipo de desarrollo
- No debe consumir más del 10% de la capacidad de trabajo del equipo
- La responsabilidad de estimar el esfuerzo previsible para cada elemento, es de las personas del equipo que previsiblemente harán el trabajo

Product Backlog ⇒ Formato

- Se prefiere la comunicación verbal o de visualización directa, a la escrita
- La pila del producto **no es un documento de requisitos**, sino una **herramienta** de referencia para el equipo
- Si se emplea formato de lista, es recomendable que al menos incluya la siguiente información para cada elemento:
  - **Identificador** único de la funcionalidad o trabajo
  - **Descripción** de la funcionalidad/requisito, denominado **historia de usuario** (*user story*)
  - Campo o sistema de **priorización**
  - Estimación del **esfuerzo** necesario

Product Backlog ⇒ Formato

- Dependiendo del tipo de proyecto, funcionamiento del equipo y la organización, pueden ser aconsejables otros campos:
  - Observaciones
  - Criterio de validación
  - Persona asignada
  - Nº de Sprint en el que se realiza
  - Módulo del sistema al que pertenece
- Es preferible **no** adoptar **formatos rígidos**. Los resultados de SCRUM no dependen de las formas, sino de la institucionalización de sus principios y la implementación adecuada a las características de la empresa y del proyecto

Artefactos: Pila de sprint

- Lista de HU, normalmente dividida en **tareas** que va a realizar el equipo en un **sprint**, para construir un **incremento**
- Para cada tarea se registra:
  - Descripción breve.
  - Persona que la tiene asignada.
  - Esfuerzo pendiente para terminarla.
- Descompone el proyecto en unidades de **tamaño adecuado** para determinar el avance a **diario**, e identificar **riesgos** y **problemas** sin necesidad de procesos complejos de gestión
- Es una herramienta de **soporte** para la **comunicación** directa del equipo

Artefactos: Pila de sprint

- ¿Por qué se añaden tareas en el Sprint Backlog?
  - Mejor seguimiento del trabajo: Las tareas ayudan al equipo a hacer un seguimiento más detallado de lo que se necesita para completar una historia de usuario.
  - Facilita la estimación: Las tareas son más fáciles de estimar y priorizar que las historias completas.
  - Transparencia: Dividiendo el trabajo en tareas claras, se mejora la visibilidad del progreso, lo que permite que el equipo gestione mejor los imprevistos.

Artefactos: Pila de sprint⇒ Condiciones

- Realizada de forma **conjunta** por todos los miembros del equipo
- Cubre **todas** las tareas **identificadas** por el equipo para conseguir el objetivo del sprint
- Sólo el **equipo** la puede **modificar** durante el sprint
- Las tareas demasiado **grandes** (de más de un día) deben **descomponerse** en otras más pequeñas
- Es **visible** para todo el equipo. Idealmente en un tablero o pared en el mismo espacio físico donde trabaja el equipo

Artefactos: Pila de sprint⇒ Formato

Tendrá que adaptarse a las características del proyecto, oficina y equipo:

- **Tablero** físico o pared
- **Hoja de cálculo**
- **Herramienta colaborativa** o de gestión de proyectos: Jira, Trello, Azure DevOps, Monday.com, Redmine...
- Incluir la siguiente información: Pila del sprint, persona **responsable** de cada tarea, **estado** en el que se encuentra y **tiempo** de trabajo que queda para completarla
- Incluir sólo la **información** estrictamente **necesaria**
- Debe servir de medio para **registrar** en cada reunión diaria del sprint, el **tiempo** que le queda a cada tarea
- Facilitar la **consulta** y la **comunicación** diaria y directa del equipo.

Durante el sprint, el equipo **actualiza** a diario en ella los **tiempos** pendientes de cada tarea para construir el **burn-down chart**

Artefactos: Incremento

- El incremento es la **parte** de producto producida en un **sprint**, y tiene como característica el estar **completamente terminada y operativa**, en condiciones de ser entregada al cliente
- **No** se deben considerar como **incremento** a **prototipos**, **módulos** o sub-módulos, ni partes pendientes de pruebas o integración
- Cada elemento de la pila del producto se refiere a funcionalidades **entregables**, **no** a **trabajos internos** como por ejemplo *diseño de la base de datos*
- Se produce un **incremento** en cada **iteración**

Artefactos: Incremento

- Sólo y como excepción en el primer sprint puede haber objetivos del tipo *contrastar la plataforma y el diseño* pueden resultar necesarios, e implican trabajos de diseño o desarrollo de prototipos para **contrastar las expectativas** de la plataforma o tecnología que se va a emplear. Pero en general:

> *"Incremento es la parte de producto realizada en un sprint potencialmente entregable: terminada y probada"*

- Si el proyecto o el sistema requiere **documentación**, o procesos de validación **y verificación** documentados, o con niveles de independencia que implican procesos con terceros, éstos también tienen que estar realizados **en el incremento, ¡¡¡y no como un incremento!!!**

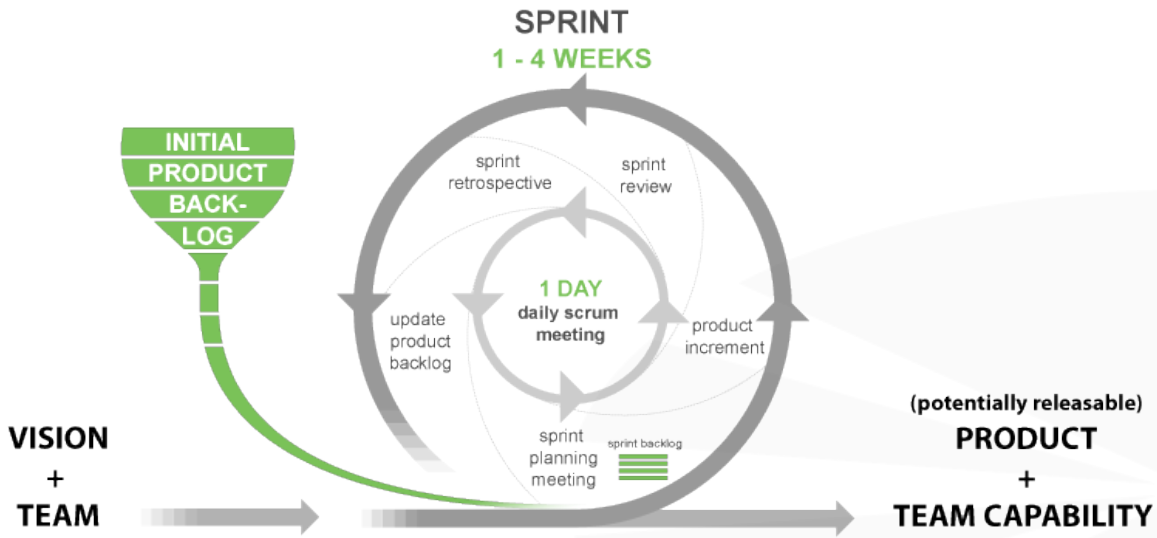
Artefactos: Incremento

> *“**Entrega continua** de funcionalidades al ritmo que se van produciendo, manteniendo un flujo continuo, sin tiempos muertos ni cuellos de botella”*
>
> *Incremento continuo*

> *“**Avance incremental** del proyecto basado en “pulsos de tiempo” o “timeboxing”*
>
> *Incremento iterativo*

Artefactos: Sprint

- Ciclo de tiempo en el que se desarrolla cada **incremento** iterativo del **producto**
- En función de las características del proyecto y el criterio del equipo, lo habitual es realizar sprints de duración no inferior a una **semana** ni mayor de un **mes**.
- Cada sprint produce un **incremento**.



Gestión de los tiempos

Gestión de los tiempos: Timeboxing

- Técnica de administración del tiempo usada en Scrum y otros modelos de gestión ágil.
- Consiste en dividir el trabajo en tareas con una asignación de tiempo limitada y definida.
- Facilita la priorización de tareas y objetivos dentro de un marco de tiempo fijo (por ejemplo, un sprint de 2 semanas).

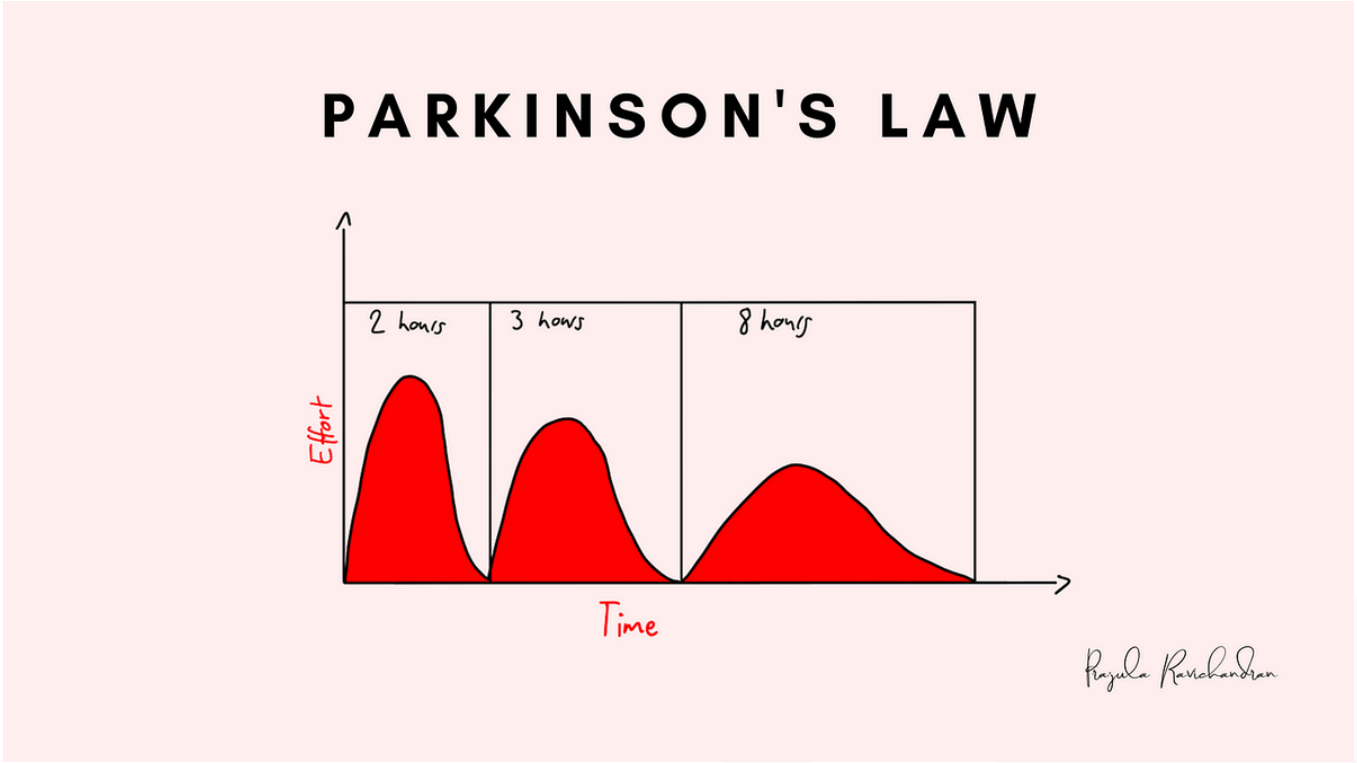
Ventajas del timeboxing

- Evita la Ley de Parkinson: El trabajo tiende a expandirse para llenar todo el tiempo disponible para su finalización.

> *"Work expands so as to fill the time available for its completion"*
>
> *Parkinson's law*

- Combate la procrastinación: Fijar plazos cortos y definidos ayuda a los equipos a concentrarse y evitar posponer tareas.
- Incrementa la eficiencia: Al tener límites claros, los equipos deben centrarse en lo más importante, evitando desviaciones innecesarias.

Parkinson's law



Ventajas del timeboxing

- Trabajar con una técnica de timeboxing requiere:
  - Objetivos claros: Definir de manera precisa qué se espera lograr en el tiempo establecido.
  - Reducción de interrupciones: Limitar distracciones y mantener el enfoque durante el periodo de trabajo fijado.
  - Retroalimentación constante: Evaluar al final de cada ciclo cómo mejorar en el siguiente.

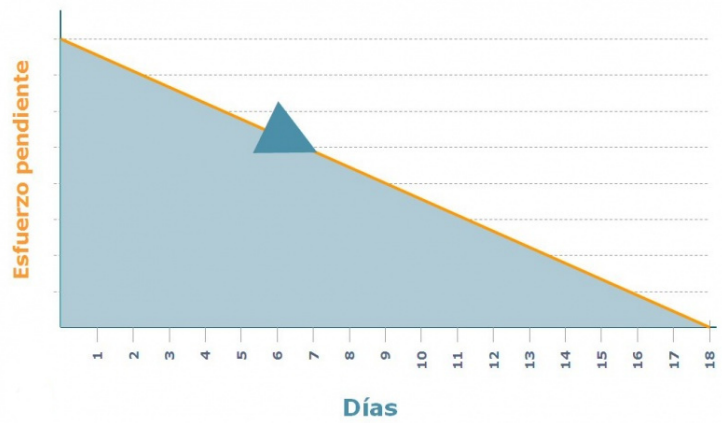
Gestión de los tiempos: Burn-down

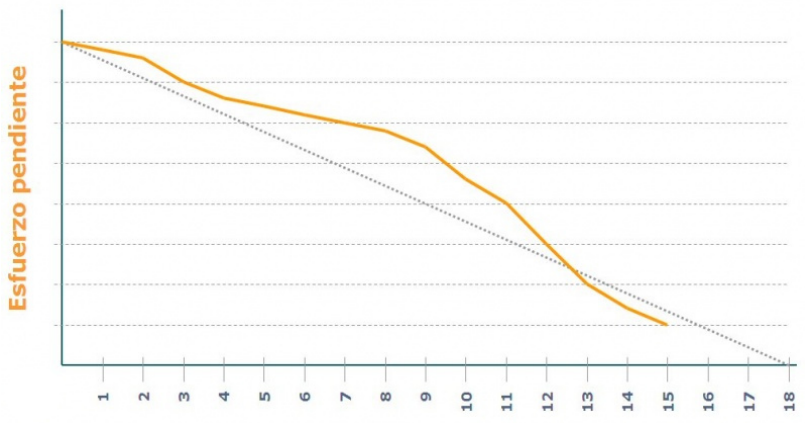
El **gráfico de avance** o ***burndown*** se actualiza por el equipo en las reuniones de seguimiento del *sprint*, para **monitorizar** el ritmo de avance, y **detectar** de forma temprana posibles **desviaciones** sobre la previsión que pudieran comprometer la entrega al final de *sprint*

- Eje **X**: los días del sprint. Eje **Y**: el trabajo que **aún falta** por hacer, en puntos o en tareas.
- Se traza primero una **línea ideal**: una recta desde todo el trabajo comprometido, el primer día, hasta 0 el último. Es el ritmo que habría que llevar si el trabajo se repartiera por igual entre los días.
- Sobre ella se dibuja la **línea real**, actualizada en cada reunión de seguimiento (a diario de ser posible).
- La lectura es la distancia entre ambas: **por encima** de la ideal, queda más trabajo del previsto y se va con retraso; **por debajo**, se va por delante.

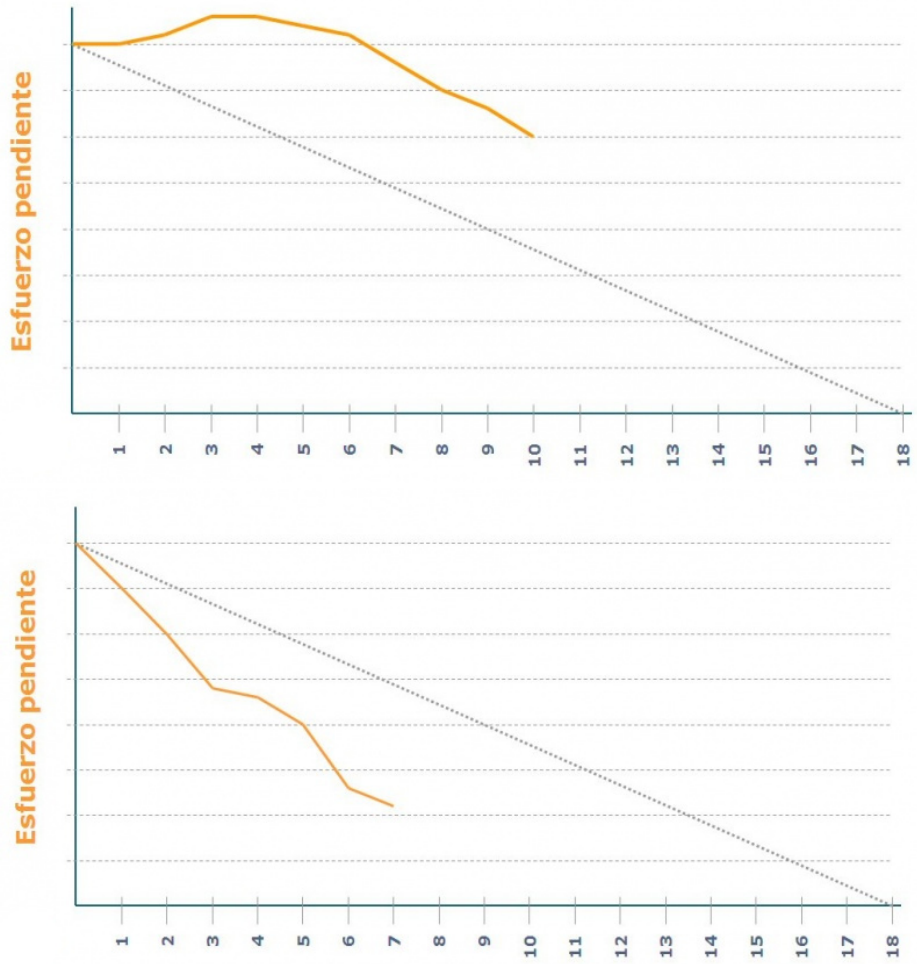
Gestión de los tiempos: Burn-down

A la izquierda, sólo la **línea ideal**: del trabajo total comprometido a 0 el último día. A la derecha se añade la **real**, que va **por encima** casi todo el sprint (retraso) y sólo la cruza al final.





Gestión de los tiempos: Burn-down



¿En qué unidad baja la línea?

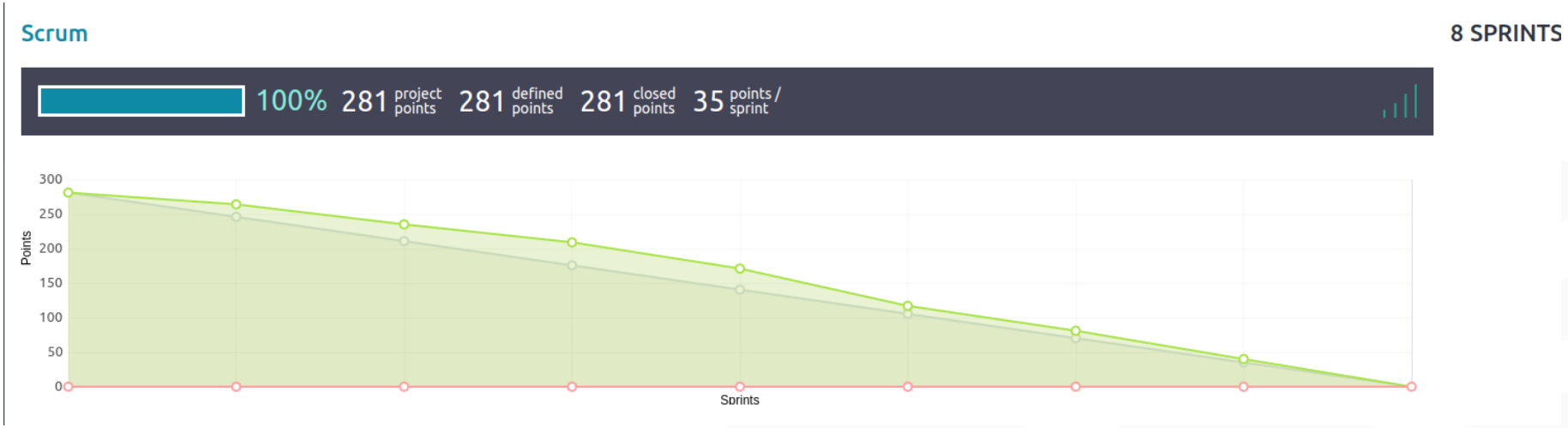
- En un burn-down de **puntos de historia** se descuentan **todos** los puntos de una historia cuando cumple la *definición de hecho*. No basta con marcarla como cerrada si aún quedan pruebas, integración u otros requisitos.
- La razón: los puntos **no se consumen poco a poco**, estiman el **tamaño relativo** de la historia. Decir *llevamos 5 de sus 15 puntos* aparenta una precisión que no se tiene.
- Si hace falta ver un avance más fino, hay dos salidas:
  - **Partir** la historia en historias más pequeñas que aporten valor por separado.
  - Llevar un burn-down de **horas pendientes** de las tareas, que sí se actualiza durante su ejecución.

Cómo leer un burn-down

- **Tramos horizontales**: con puntos son normales entre cierres; si duran mucho, las historias son **demasiado grandes**. Con horas significan trabajo **parado o bloqueado**.
- **Caída en picado el último día**: todo se cierra de golpe, y el gráfico no sirvió para anticipar nada.
- **Subidas**: ha entrado trabajo nuevo en mitad del sprint, o una estimación se quedó corta.
- Terminar **por encima de 0** significa que el sprint no se completó; terminar muy pronto, que quizá se comprometió poco trabajo.
- Lo valioso no es acabar en 0, sino **ver la desviación con tiempo** para reaccionar dentro del sprint.

Gestión de los tiempos: Project Burn Down Chart

- Existe un Burn Down a nivel de todo el proyecto, llamado Project Burn Down Chart
- En lugar de enfocarse únicamente en un sprint, este gráfico abarca todo el proyecto y refleja el progreso acumulado de todos los sprints hasta la fecha. El eje X representa los sprints, y el eje Y muestra el trabajo total restante



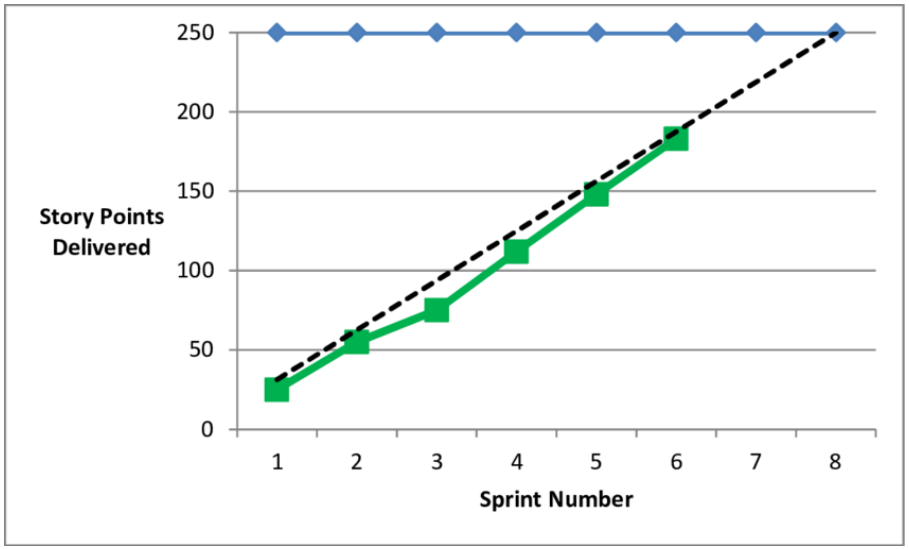
Gestión de los tiempos: Burn-up

El **gráfico del producto** o ***burnup*** es una herramienta de planificación del propietario de producto: en lugar del trabajo que queda, dibuja el trabajo **ya completado**.

- Eje **X**: el tiempo, medido en sprints. Eje **Y**: puntos de historia, tareas o funcionalidades.
- La diferencia clave es que tiene **dos líneas**: una que sube con lo completado y otra que marca el **alcance total** del proyecto. El proyecto termina cuando se tocan.
- Eso separa dos cosas que el burn-down confunde. Si la distancia al objetivo no baja, el burn-down sólo muestra una línea plana, sin decir si el equipo no avanza o si ha **entrado trabajo nuevo**.
- En el burn-up se distingue: la línea de completado sigue subiendo, pero la del alcance ha subido también. El **crecimiento del alcance** queda a la vista.

Gestión de los tiempos: Burn-up

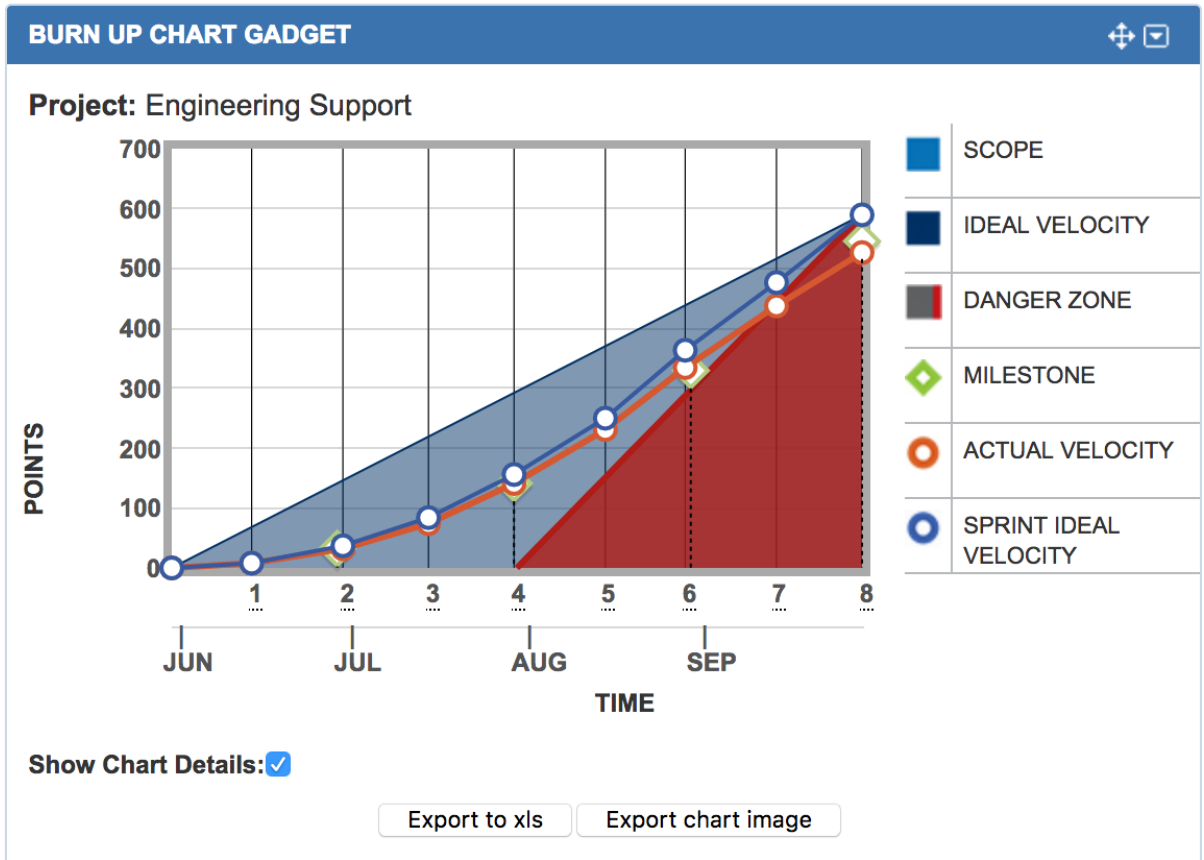
El gráfico no trae leyenda. Las tres líneas son:



- **Azul horizontal**: el alcance total, 250 puntos; plana porque aquí no cambia.
- **Verde**: puntos entregados acumulados. **Discontinua**: el ritmo necesario para llegar a 250 en 8 sprints.
- El verde va por debajo de la discontinua: a ese ritmo no se cubre el alcance.

Gestión de los tiempos: Burn-up

Así se ve en una herramienta real, que desglosa el mismo gráfico en más series:



Gestión de los tiempos: Unidades de trabajo

- Antes de ver cómo se estima en ágil conviene saber cómo se hacía tradicionalmente. Durante décadas la industria del software estimó con **modelos paramétricos**: fórmulas calibradas sobre proyectos pasados que, a partir del **tamaño** previsto del producto, devuelven **esfuerzo** y **plazo**.
- Un trabajo puede dimensionarse midiendo el **producto** que se construye, con los tradicionales **puntos de función**, o el **tiempo** que cuesta realizarlo.
- El modelo de referencia es **COCOMO** (**CO**nstructive **CO**st **MO**del) [Boehm, *SE Economics* 1981; COCOMO II 2001], de base empírica.
- COCOMO I asumía desarrollo en cascada; COCOMO II admite distintas metodologías de programación, y se asienta en los 90 como el principal método de estimación.

Gestión de los tiempos: Unidades de trabajo

- Un factor para calcular el esfuerzo en COCOMO son las **líneas de código**, para estimar dicho número se usan los **puntos de función**
- Para calcular los puntos de función hay que contar:
  - Número de **entradas** de usuario
  - Número de **salidas** para usuario
  - Número de **consultas** de usuario
  - Número de **ficheros**
  - Número de **interfaces** externas

Gestión de los tiempos: Unidades de trabajo

$$FP = \text{conteo total} \times \left[0.65 + 0.01 \times \sum_i F_i\right]$$

parámetro	sencillo	promedio	complejo
# user inputs	3	4	6
# user outputs	4	5	7
# user inquiries	3	4	6
# files	7	10	15
# external interfaces	5	7	10

- El **conteo total** sale de la tabla: cada elemento se multiplica por el peso de su columna y se suman todos.
- Convertir después los FP en número de líneas depende de un factor propio de cada lenguaje de programación.

Gestión de los tiempos: Unidades de trabajo

- El conteo de la tabla mide **qué hace** el sistema, no lo **exigente** que es construirlo: dos sistemas con las mismas funciones no cuestan igual si uno es distribuido y en tiempo real y el otro no.
- El corchete corrige eso. Se puntúan **14 características técnicas** de 0 (irrelevante) a 5 (crítica): comunicaciones por red, entrada en tiempo real, complejidad de la lógica, reusabilidad…
- Las 14 notas se suman y el corchete las convierte en un **multiplicador** del conteo:

$\sum_i F_i$	corchete	efecto
0 (nada exigente)	0.65	-35%
35 (término medio)	1.00	sin cambio
70 (todo crítico)	1.35	$+35\%$

- El 0.65 y el 0.01 no significan nada por separado: son las constantes que producen esa ventana de $\pm 35\%$.

Gestión de los tiempos: Unidades de trabajo

- **Ejemplo**: un sistema con 12 entradas, 8 salidas, 10 consultas, 5 ficheros y 2 interfaces externas, todas de complejidad promedio.
- Conteo total, con los pesos de la columna *promedio*: $12{\cdot}4 + 8{\cdot}5 + 10{\cdot}4 + 5{\cdot}10 + 2{\cdot}7 = 192$
- Si el sistema es del montón en las 14 preguntas ($F_i = 2.5$ de media, luego $\sum_i F_i = 35$), el ajuste vale $0.65 + 0.35 = 1$ y por tanto $FP = 192$.
- Con un lenguaje de unas 50 líneas por punto de función: $192 \times 50 = 9600$ líneas, es decir 9.6 KLOC.
- COCOMO básico, modo orgánico: $2.4{\cdot}9.6^{1.05} \approx 26$ personas-mes y $2.5 \cdot 26^{0.38} \approx 9$ meses, unas 3 personas.

Gestión de los tiempos: Unidades de trabajo

- El resultado: una fecha y un coste concretos antes de escribir una sola línea de código.
- Conviene mirar lo que ha costado obtenerlo:
  - Hay que tener el sistema **completamente especificado** para contar entradas, salidas, consultas, ficheros e interfaces.
  - Hay que responder a 14 preguntas sobre un producto que todavía no existe.
  - Hay que fijar el lenguaje de antemano, porque el factor de conversión depende de él.
- La estimación se hace **una sola vez**, al principio, que es justo el momento de **máxima incertidumbre**, y cualquier cambio de requisitos la invalida.
- El enfoque ágil invierte el planteamiento: estimaciones baratas y groseras, revisadas cada sprint con datos reales. Es lo que veremos a continuación.

Entonces, ¿COCOMO ya no sirve?

- No es *antiguo frente a moderno*: COCOMO II se sigue manteniendo, aunque su ámbito es hoy mucho más estrecho.
- El enfoque predictivo encaja cuando hay que comprometerse **antes** de empezar:
  - **Precio cerrado**: licitaciones y contratación pública.
  - **Requisitos congelados por normativa**: sistemas certificables (aeroespacial, dispositivos médicos).
  - **Decisión de cartera**: cuando aún no hay equipo al que medirle la velocidad.
- Su límite: calibrado sobre proyectos históricos, fuera de esa muestra aparenta una precisión que no tiene.
- No compiten: COCOMO responde *¿cuánto costará el proyecto?* y la velocidad *¿cuánto cabe en el próximo sprint?* El error habitual es usar uno para la pregunta del otro.

Medición del trabajo

- Medir sirve para responder a dos preguntas distintas:
  - ¿Cuánto trabajo se ha **completado**? Es un hecho observable.
  - ¿Cuánto trabajo **queda**? Es una estimación, y por tanto incierta.
- Medir lo ya hecho no entraña especial dificultad; el problema está siempre en lo que queda.
- Las unidades posibles se refieren al **producto** construido (líneas de código, puntos de función) o a los **recursos** empleados (tiempo de trabajo, coste).
- Ninguna resulta cómoda para planificar: medir el producto exige un análisis previo demasiado pesado para el ritmo ágil, y medir recursos hace que la estimación dependa de quién ejecute la tarea.
- Por eso en ágil se estima en **puntos de historia** y la conversión a tiempo se deja en manos de la velocidad observada.

Estimación del trabajo

- El trabajo de una historia de usuario **no se puede prever de forma exacta**: no tiene una única solución posible, y la capacidad y las circunstancias de cada persona varían.
- La respuesta ágil no es estimar mejor, sino **estimar barato y corregir a menudo**:
  - Estimar por **comparación** con historias ya completadas, no desde cero.
  - Usar el **juicio del equipo** (p. ej. planning poker) en vez de la opinión de una sola persona.
  - **Descomponer** las historias que no caben en un sprint y las tareas que superan un día de trabajo.
  - **Recalibrar** cada sprint con la velocidad realmente observada.
- No se busca **precisión** historia a historia, sino **consistencia**: con un criterio estable, los errores individuales se compensan y la velocidad se vuelve predictiva.

Tiempo real y tiempo ideal

- **Tiempo real**: el que marca el calendario. Es la jornada laboral completa, con sus reuniones, correos, interrupciones y cambios de contexto.
- **Tiempo ideal**: el que costaría la tarea si sólo se trabajase en ella, sin interrupciones ni asuntos ajenos y con plena disponibilidad.
- La diferencia no es un defecto del equipo: una jornada de 8 horas reales rinde bastante menos de 8 horas ideales, y la proporción depende del entorno de trabajo.
- Por eso el tiempo ideal **no se usa para dar fechas**. Sirve para expresar **cuánto trabajo** tiene una tarea, no cuándo estará terminada.

Del tiempo ideal a los puntos de historia

- El tiempo ideal mide el esfuerzo mejor que el tiempo real, pero sigue siendo una medida **personal**: la misma tarea puede ser un día ideal para quien conoce el código y tres para quien no.
- Scrum evita ese problema estimando **tamaño relativo** en lugar de duración: los **puntos de historia**.
- Un punto de historia **no es una unidad de tiempo**, y por tanto no responde por sí solo a *¿cuándo estará?*.
- Esa pregunta se contesta aparte, con la **velocidad** del equipo, que cerrará este bloque: se **estima en puntos** y se **mide en tiempo real**, y la velocidad conecta ambas cosas.

¿Qué es exactamente un punto de historia?

- Un punto de historia es una medida **relativa** y **sin unidades** del tamaño de una historia: resume volumen de trabajo, complejidad e incertidumbre, y sólo significa algo **dentro de un equipo**.
- Pero toda medida relativa necesita una referencia, y la primera historia no tiene con qué compararse. De ahí el arranque habitual:
  - Fijar un ancla provisional: **1 punto $\approx$ 1 hora de trabajo ideal**, o una historia pequeña y conocida que se declara *la historia de 1 punto*.
  - Estimar el resto por comparación con ella, sobre una escala gruesa (1, 2, 3, 5, 8, 13…).
- Tras dos o tres sprints hay historias terminadas y un **ritmo real medido**: desde ahí se estima comparando con lo ya hecho y **el ancla temporal se abandona**.
- Si no se abandona, los puntos son horas encubiertas y acaban leyéndose como un **compromiso de plazo**.

La escala en la práctica

- Una escala típica de un equipo:
  - Pequeña: crear un perfil de usuario, **2** puntos.
  - Mediana: calcular estadísticas de uso de un grupo de usuarios, **8** puntos.
  - Grande: integrar un módulo de informes con filtros y exportación, **15** puntos.
- Lo que importa no es el número absoluto, sino que la **relación** entre historias sea coherente: que la mediana sea de verdad unas cuatro veces la pequeña.
- Esa coherencia exige un **criterio común**: todo el equipo asigna puntos con la misma lógica, y la mantiene a lo largo del proyecto. Si cada persona usa su propia escala, los puntos dejan de ser comparables entre sí.

Velocidad en la gestión ágil

- La **velocidad** en Scrum se define como la cantidad de trabajo realizada en un sprint o un ciclo de tiempo determinado.
- Se expresa comúnmente como la suma de puntos de historia completados por el equipo durante un sprint.
- La velocidad ayuda a predecir la capacidad de trabajo del equipo y a planificar sprints futuros.
- Fórmula básica (cantidad de trabajo realizada por unidad de tiempo):

$$Velocidad = \frac{Trabajo\ completado}{Tiempo\ empleado}$$

- Es la pieza que convierte puntos en fechas: sprints restantes $\approx$ puntos pendientes/velocidad.

La velocidad en un caso concreto

Un equipo de 5 personas, con sprints de 2 semanas, ha completado:

Sprint	1	2	3	4
Puntos completados	18	26	22	24

- Velocidad media: $(18+26+22+24)/4 = \mathbf{22{,}5}$ puntos por sprint.
- Si en la pila de producto quedan **180** puntos: $180/22{,}5 = 8$ sprints, es decir unas **16 semanas**.
- Pero la velocidad real osciló entre 18 y 26. Haciendo la cuenta con los extremos salen **entre 7 y 10 sprints**. La previsión honesta es ese rango, no el 8.

Dos advertencias sobre la velocidad

- **No es un objetivo**. Es una medida descriptiva de lo que el equipo hizo. Si se usa para presionar o para evaluar, el equipo infla las estimaciones, la velocidad sube y el número deja de medir nada.
- **No sirve para comparar equipos**. Los puntos son relativos a la escala de cada equipo, así que 30 puntos de uno y 30 de otro no significan lo mismo. Sólo tiene sentido comparar un equipo consigo mismo a lo largo del tiempo.
- Por eso hacen falta varios sprints antes de que la velocidad sea útil: con uno o dos datos, la media no dice nada.

Métodos de avance: Iterativo vs. Continuo

- Para mantener un ritmo de avance continuo en el **tiempo**, el desarrollo ágil emplea dos tácticas posibles: incremento **iterativo**, e incremento **continuo**
- Incremento iterativo: El equipo trabaja en sprints definidos, cada uno con un conjunto de funcionalidades específicas.
  - El avance a través de incrementos **iterativos** mantiene el **ritmo** apoyándose en pulsos de **sprints**.
  - Ejemplo: En un sprint de 2 semanas, el equipo completa 30 puntos de historia.
- Incremento continuo: El flujo de trabajo es constante y continuo, manteniendo un flujo constante sin puntos muertos ni cuellos de botella.
  - La velocidad se mide en puntos por día/semana/mes...
  - Ejemplo: El equipo completa 10 puntos de historia por semana en un flujo de trabajo continuo.

¿Y quién decide los puntos? -> Planning Poker

Planning poker es una técnica para calcular una estimación basada en el **consenso**, en su mayoría utilizada para estimar el esfuerzo o el tamaño relativo de las tareas de desarrollo de software

Es una variación del método Wideband Delphi. Es utilizado **comúnmente** en el desarrollo **ágil** de software

El método fue descrito por primera vez por James Grenning en 2002 y más tarde se volvería más popular y comercial por Mike Cohn con el libro **Agile Estimating and Planning**

Planning Poker

El póker de planificación está basado en una **lista de características** para ser entregadas y una **baraja de cartas**

La lista de características, por lo general una lista de **historias** de usuario, describen un software que necesita ser desarrollado

¿De dónde salen estas características?

Planning Poker

Las **cartas** en el mazo están **numeradas**. Un mazo típico contiene tarjetas mostrando la secuencia de **Fibonacci** incluyendo un cero: 0, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89

Otros mazos utilizan progresiones similares, la razón de utilizar la secuencia de Fibonacci es reflejar la **incertidumbre** inherente en la estimación

Un mazo que se encuentra en el mercado utiliza la siguiente secuencia: 0, 1/2, 1, 2, 3, 5, 8, 13, 20, 40, 100, y adicionalmente dos tarjetas, una con signo de interrogación (?) y otra con signo de infinito (∞), que pueden ser usadas para declarar completa incertidumbre o desconocimiento de la característica

Planning Poker

- **Crecimiento no lineal:** A medida que las tareas son más grandes, aumenta la incertidumbre. Fibonacci refleja eso: la diferencia entre 5 y 8 es menor que entre 13 y 21.
- **Evita debates excesivos:** Al haber "saltos" entre números, se fomenta la discusión solo cuando las percepciones de esfuerzo realmente difieren.
- **Facilita la priorización:** Es más fácil identificar tareas desproporcionadas y dividirlas si caen en números altos como 13 o 21.

Planning Poker

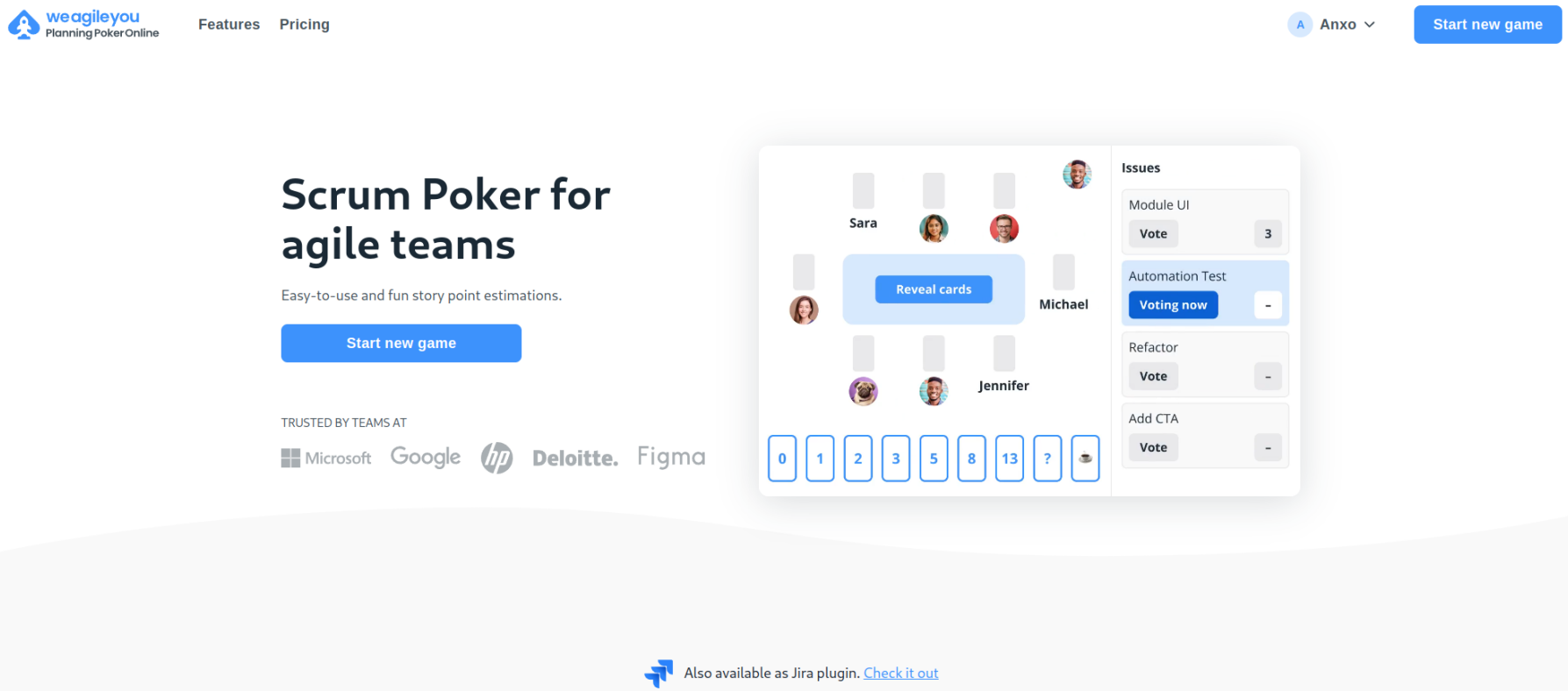
En la reunión de la estimación a cada **estimador** se le da un mazo completo de tarjetas.

La reunión prosigue de la siguiente manera:

1. Un **moderador**, que no jugará, preside la reunión, apoyado y asesorado por el Gestor del Proyecto
2. El **desarrollador** con más conocimiento de una determinada característica proporciona una breve **introducción** sobre la misma. El equipo tiene la oportunidad de hacer **preguntas** y discutir para **aclarar** los supuestos y riesgos. Un resumen de la discusión es registrado por el Gestor del Proyecto
3. Cada persona coloca una tarjeta boca abajo que representa su **estimación**. Las unidades utilizadas pueden ser variadas y definidas previamente. Pueden ser **días** de **duración**, días **ideales** o **puntos** de la historia. Durante el debate, los números **no** debe ser **mencionados** en absoluto

Planning Poker

1. Todo el mundo **muestra** sus tarjetas de forma **simultánea**
2. A las personas con estimaciones altas y bajas se les da un tiempo para ofrecer su **justificación** para la estimación y la **discusión** continúa
3. Se repita el proceso de cálculo hasta que se alcance un consenso. El **programador** que probablemente tenga el **entregable** tiene una gran **parte** del voto de consenso, aunque el **moderador** puede **negociar** el consenso
4. Se puede utilizar un **reloj** de arena para asegurar que el debate sea **estructurado**, el moderador o el Gestor del Proyecto podrá en cualquier punto terminar el reloj y cuando se acaba toda discusión debe cesar y se juega **otra ronda de póker**


weagileyou
Planning Poker Online
Features
Pricing
A
Anxo
Start new game
Scrum Poker for agile teams
Easy-to-use and fun story point estimations.
Start new game
TRUSTED BY TEAMS AT
Microsoft
Google
Deloitte.
Figma
Sara
Michael
Jennifer
Reveal cards
Issues
Module UI
Vote
3
Automation Test
Voting now
-
Refactor
Vote
-
Add CTA
Vote
-
0
1
2
3
5
8
13
?
Also available as Jira plugin. Check it out

Reuniones

Dos vocabularios para lo mismo

Tanto este tema como la bibliografía alternan los nombres en inglés y en castellano. Son equivalentes:

Inglés	Castellano
Product Owner	Propietario del producto
Development Team	Equipo de desarrollo
Product Backlog	Pila del producto
Sprint Backlog	Pila del sprint
Sprint Goal	Objetivo del sprint
Increment	Incremento
Definition of Done	Definición de hecho

En las herramientas y en el mercado laboral predomina la forma inglesa; *Scrum Master* y *sprint* no suelen traducirse.

Eventos en Scrum

- Scrum incluye una serie de eventos clave que estructuran el trabajo del equipo durante el desarrollo de un producto.
- Estos eventos ayudan a coordinar, planificar, revisar y mejorar el trabajo realizado por el equipo.
- Los principales eventos son:
  - **Sprint Planning** (Planificación del Sprint)
  - **Daily Scrum** (Reunión Diaria)
  - **Sprint Review** (Revisión del Sprint)
  - **Sprint Retrospective** (Retrospectiva del Sprint)

Sprint Planning

- El evento de **Sprint Planning** marca el inicio de cada sprint.
- El equipo se reúne para discutir y definir **qué** trabajo se hará durante el sprint y **cómo** se completará.
- Los principales temas que se abordan son:
  - ¿Qué se puede entregar al final del sprint? Basado en la prioridad del **Product Backlog** y la capacidad del equipo.
  - ¿Cómo se hará el trabajo seleccionado? El equipo desglosa las historias de usuario seleccionadas en tareas más pequeñas y detalladas.
- El resultado es el **Sprint Backlog**, una lista de las tareas y objetivos a completar durante el sprint.
- La planificación del sprint es una colaboración entre el Product Owner y el equipo de desarrollo.

- **Precondiciones**:
  - La organización tiene claros los **recursos** disponibles para llevar a cabo el sprint
  - Ya están listos los elementos prioritarios de la pila del producto, de forma que ya tienen un nivel de **detalle** suficiente y una **estimación** previa del trabajo que requieren
  - El equipo tiene un **conocimiento** de las **tecnologías** empleadas, y del negocio del producto suficiente para realizar **estimaciones** basadas en juicio de expertos, y para **comprender** los **conceptos** del **negocio** que expone el propietario del producto

Sprint Planning

- **Entradas**:
  - La **pila** del producto
  - El **producto desarrollado** hasta la fecha en los incrementos anteriores (excepto si se trata del primer sprint)
  - Dato de la **velocidad** o rendimiento del **equipo** en el último sprint, que se emplea como criterio para estimar la cantidad de trabajo que es razonable suponer para el próximo sprint
  - **Circunstancias** de las condiciones de **negocio** del cliente y del escenario tecnológico empleado
- **Resultados**:
  - **Pila** del sprint
  - **Duración** del sprint y **fecha** de la reunión de revisión
  - **Objetivo** del sprint

Sprint Planning

Sus dos partes:

- **Primera parte: Qué se entregará al terminar el sprint**:
  - El propietario del producto presenta la **pila de producto**, exponiendo los requisitos de mayor **prioridad** que necesita y que prevé que se podrán desarrollar en el siguiente sprint
  - Si la pila del producto ha tenido **cambios** significativos desde la anterior reunión, explica las **causas** que los han ocasionado
  - El objetivo es que todo el equipo conozca las **razones y los detalles** con el nivel suficiente para comprender el trabajo del sprint
  - Para ello el equipo debe realizar las **preguntas** y solicitar las **aclaraciones** necesarias así como proponer **sugerencias**, modificaciones y soluciones alternativas
  - Tras reordenar y replantear las funcionalidades de la pila del producto, el equipo define el **objetivo del sprint**: una frase que sintetiza qué valor se le va a entregar al cliente

El objetivo del sprint como filtro

- El objetivo no sirve solo para resumir el sprint: es lo que le da al equipo un **criterio para decir que no**. Sin él, cualquier petición que llegue parece igual de legítima.
- Si a mitad de sprint aparece una historia que no contribuye al objetivo, el equipo la deja para el siguiente. Es la **reducción de interrupciones** que vimos en el *timeboxing*.
- El matiz: lo que no cambia es el **objetivo**; el **alcance** sí se renegocia entre propietario y equipo. Una historia que sí sirve al objetivo puede entrar, pero a cambio de que salga otra: la capacidad está fija.
- En el burn-down se ve: meter trabajo sin sacar nada produce una **subida** de la línea a mitad de sprint.

Reuniones: Sprint planning

- **Segunda parte: Cómo se conseguirá hacer el incremento**:
  - El **equipo desglosa** cada funcionalidad en **tareas**, y estima el tiempo para cada una de ellas, componiendo así las tareas que forman la pila del sprint
  - En este desglose, el equipo tiene en cuenta los **elementos de diseño y arquitectura** que deberá incorporar el sistema
  - El equipo establece cuáles van a ser las tareas para los primeros días del sprint, y se las **autoasignan** tomando como criterios sus **conocimientos**, intereses y una distribución homogénea del trabajo
  - Esta segunda parte debe considerarse como una **reunión del equipo**, en la que deben estar todos sus miembros, y ser ellos quienes descompongan estimen y asignen el trabajo
  - El rol del propietario del producto es atender a dudas y comprobar que el equipo comprende y comparte su objetivo
  - El Scrum Master actúa de **moderador** de la reunión

Sprint Planning

El Scrum Master debe

- **Realizar** esta reunión antes de cada sprint
- Asegurar que la **pila del producto** llega adecuadamente preparada por el propietario
- Ayudar a mantener el **diálogo** entre el propietario y el equipo
- Asegurar que se llegue a un **acuerdo** entre el propietario del producto y el equipo respecto de lo que incluirá el **incremento**
- Ayudar al equipo a **comprender** la visión de negocio del cliente
- Asegurar que la **descomposición** y la estimación son realistas, y que se han considerado las tareas de análisis, investigación o apoyo
- Asegurar que al final de la reunión están objetivamente determinados los **resultados** vistos antes: elementos seleccionados, objetivo, pila del sprint estimada, duración y fecha de revisión

Daily Scrum: Reunión Diaria

- Evento diario de **15 minutos como máximo**, siempre a la misma hora y en el mismo lugar. Se recomienda hacerla de pie, para que no se alargue.
- Su objetivo es revisar el progreso hacia el **objetivo del sprint** y ajustar el plan del día.
- Formato flexible; lo habitual es que cada miembro responda a tres preguntas:
  - ¿Qué hice ayer que contribuyera al sprint?
  - ¿Qué haré hoy?
  - ¿Hay algún obstáculo que me impida avanzar?
- **No es una reunión de estado** para el propietario del producto ni el Scrum Master, sino el momento en que el equipo se coordina y se auto-organiza.

Para qué sirve la Daily

- **Detectar bloqueos pronto**, antes de que comprometan el sprint, y poder corregir el rumbo a diario en vez de descubrirlo al final.
- **Mantener el foco** en el objetivo del sprint y hacer visible el progreso de cada uno ante el resto del equipo.
- **Sustituir** las reuniones largas de seguimiento por un intercambio corto y concreto.
- Es una reunión **técnica y del equipo**: los stakeholders externos solo participan si el propio equipo los necesita para desbloquear algo.

Sprint Review

- Al final de cada sprint se celebra la Sprint Review (revisión del sprint), donde el equipo presenta lo que ha sido completado y lo que no.
- Los objetivos principales de esta reunión son:
  - Demostrar el incremento: El equipo muestra al Product Owner y otros stakeholders el trabajo que ha sido completado.
  - Recoger feedback: Los interesados (stakeholders) proporcionan retroalimentación sobre el incremento, lo que puede influir en los futuros sprints.
  - Revisar el Product Backlog: Se ajusta el **Product Backlog** en función de lo que se ha completado y las nuevas prioridades.
- La Sprint Review no es una presentación formal, sino una discusión colaborativa para inspeccionar el trabajo y adaptarlo según sea necesario.

Reuniones: Revisión del Sprint

- No debe durar más de 4 horas, en el caso de revisar sprints largos (1 hora para sprints cortos)
- Tiene como objetivos
  - El propietario del producto comprueba el **progreso** del sistema
  - El propietario del producto identifica las **funcionalidades** que se pueden considerar **hechas** y las que no
  - Al **ver y probar** el incremento, el propietario del producto, y el equipo en general obtienen **feedback** relevante para revisar la pila del producto
  - Otros ingenieros y programadores de la empresa también pueden asistir para conocer **cómo trabaja la tecnología** empleada

Reuniones: Revisión del Sprint

- Precondiciones
  - Se ha **concluido** el sprint
  - **Asiste** todo el equipo de desarrollo, el propietario del producto, el Scrum Master y todas las personas implicadas en el proyecto que lo deseen
- Entradas
  - Incremento finalizado
- Resultados
  - **Feedback** para el propietario del producto: hito de seguimiento de la construcción del sistema, e información para mejorar el valor de la visión del producto
  - **Convocatoria** de la reunión de planificación del siguiente sprint

Reuniones: Revisión del Sprint

- Informal -> Están **prohibidas** las **presentaciones** gráficas
- Según las características del proyecto puede incluir también **documentación** de usuario, o técnica
- Su misión **no** es la toma de **decisiones** ni la **crítica** del incremento
- Los pasos recomendados son:
  - El equipo expone el **objetivo** del sprint, la lista de **funcionalidades** que se incluían y las que se han **desarrollado**
  - El equipo hace una introducción general del **sprint** y **demuestra** el **funcionamiento** de las partes construidas
  - Se abre un turno de **preguntas** y **sugerencias**. Esta parte genera información valiosa para que el propietario del producto y el equipo en general, puedan mejorar la visión del producto
  - El Scrum Master, de acuerdo con las agendas del propietario del producto y el equipo, **cierra la fecha** para la reunión de preparación del siguiente sprint

Reuniones: Backlog Grooming

En ciertas ocasiones, tambien existe la llamada revisión de la pila del producto y refinamiento de la pila del producto

- Es la reunión del **propietario** del producto con el **equipo** para mantener **actualizada** la pila del **producto**.
- Se plantea con un **tiempo** máximo **prefijado** (habitualmente una o dos horas) y un objetivo que puede implicar cualquiera de las siguientes actividades:
  - **Incorporación** de nuevas historias de usuario
  - **División** de historias de usuario que resultan demasiado grandes
  - **Replanteamiento** de historias ya definidas
  - **Estimación** de las historias de usuario
- Sirve para **orientar al propietario de producto** del tamaño y estimación previa de las historias que va incorporando a la pila del producto e **incrementar el nivel de concreción** de las historias que por su prioridad se van acercando hacia el próximo sprint

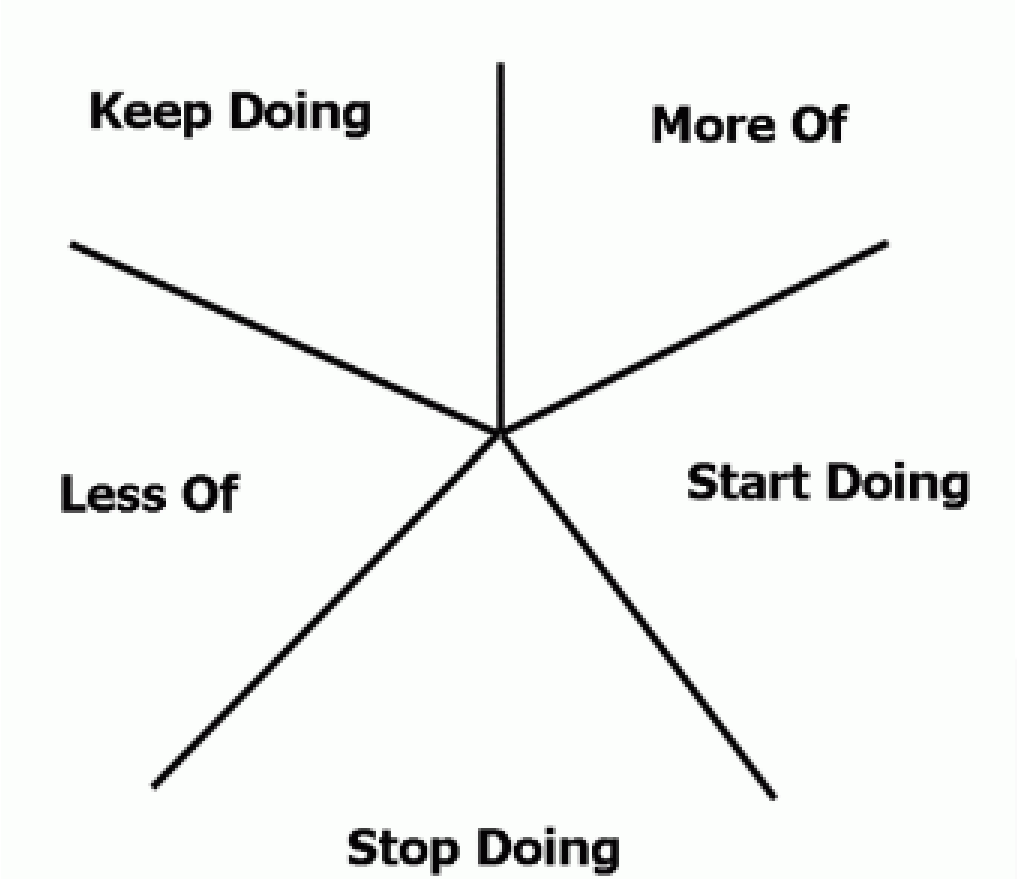
Sprint Retrospective

- La **Sprint Retrospective** se celebra al final del sprint, después de la Sprint Review. No debe confundirse con ella: la Review mira el **producto**, la Retrospective mira el **proceso**.
- El objetivo es que el equipo reflexione sobre qué funcionó y qué puede mejorar en los siguientes sprints. Las preguntas clave son:
  - *¿Qué salió bien en el sprint?*
  - *¿Qué no salió tan bien?*
  - *¿Qué podemos hacer de manera diferente en el próximo?*
- Participa el **equipo Scrum**: equipo de desarrollo, Scrum Master y propietario del producto. Los clientes y otros *stakeholders* **no** asisten, porque la reunión depende de que se pueda hablar con franqueza de lo que salió mal.
- Debe terminar con **acuerdos concretos**, que se implementan en los sprints siguientes. Una retrospectiva sin acciones es una queja colectiva.

Técnicas para facilitar la Retrospectiva

- Existen varias técnicas para estructurar la discusión en la Sprint Retrospective. Algunas populares incluyen:
  - **Start, Stop, Continue**: Qué cosas comenzar, dejar de hacer o continuar.
  - **Mad-Sad-Glad**: Identificar situaciones que generaron frustración, tristeza o satisfacción en el sprint.
  - **Timeline**: El equipo dibuja una línea de tiempo del sprint y señala los momentos altos y bajos.
- El Scrum Master facilita la sesión, asegurándose de que sea un entorno seguro para la retroalimentación abierta y constructiva.
- La retrospectiva debe culminar con una lista de acciones concretas que mejoren el rendimiento del equipo en el próximo sprint.

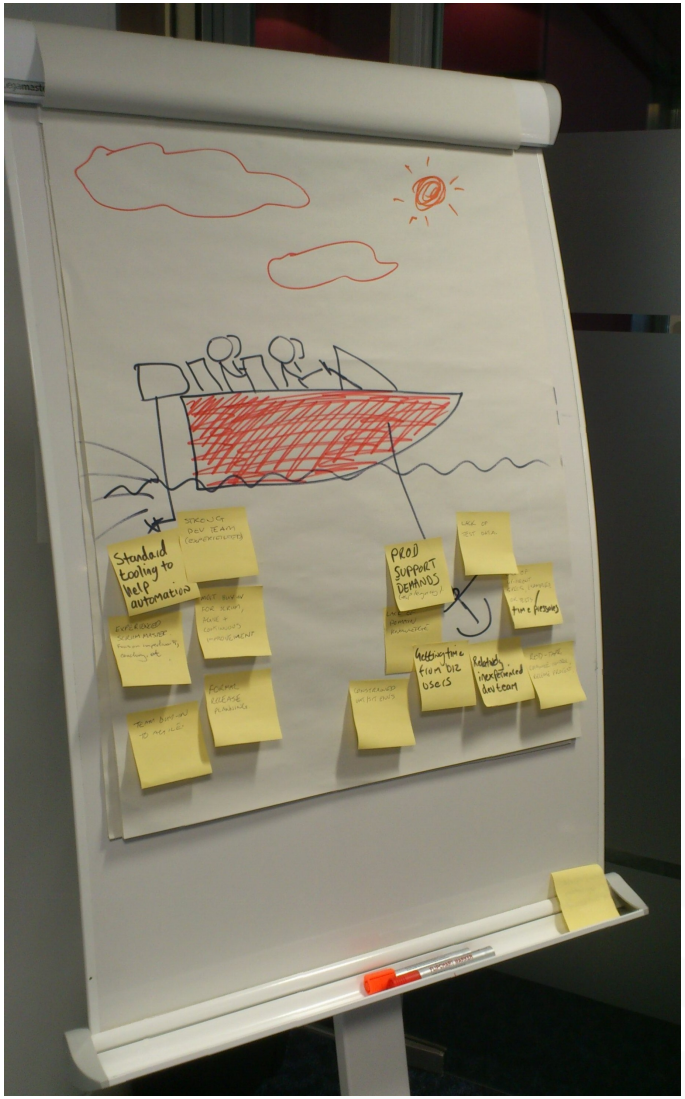
Técnicas de Retrospective: Estrella de mar



Técnicas de Retrospective: Estrella de mar



Técnicas de Retrospective: Barco



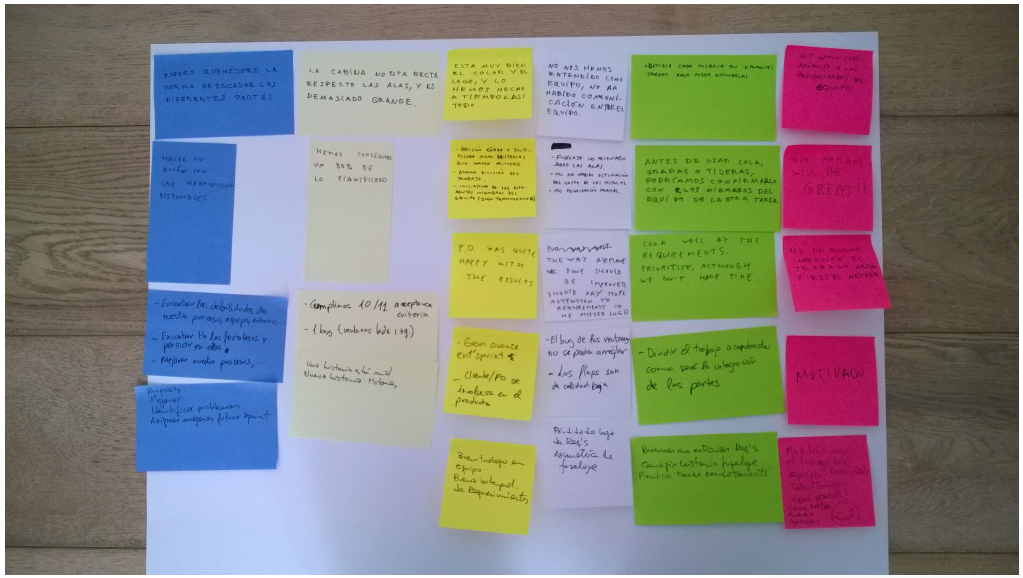
Técnicas de Retrospective: Barco

- La técnica del Barco es una herramienta visual para identificar las fuerzas que impulsan o detienen el progreso del equipo.
- Categorías clave:
  - Viento a favor (Boosters): Factores que aceleran el avance del equipo hacia la meta.
  - Anclas (Blockers): Obstáculos o problemas que ralentizan el progreso.
  - Meta (Isla): El objetivo o producto final al que el equipo se dirige.
  - Tormentas (Riesgos): Amenazas que pueden surgir y afectar el proyecto en el futuro.
  - Recursos/Boyas de Salvamento: Ayudas externas o herramientas que pueden ayudar al equipo.

Técnicas de Retrospective: Seis sombreros

La técnica de los Seis Sombreros para Pensar, desarrollada por Edward de Bono, es una herramienta de pensamiento grupal que ayuda a los equipos a ver un problema desde diferentes perspectivas. La técnica se basa en el uso de seis sombreros imaginarios, cada uno con un color, que representan diferentes modos de pensar

Técnicas de Retrospective: Seis sombreros



Técnicas de Retrospective: Seis sombreros

- **Azul**: para discutir los objetivos de la reunión y apuntarlos en la pizarra
- **Blanco**: el equipo señala todo aquello que desde la última retrospectiva pueda servir como información y hechos objetivos
- **Amarillo**: aspectos positivos ocurridos desde la última reunión
- **Negro**: aspectos negativos ocurridos desde la última reunión
- **Verde**: propuestas para solucionar problemas, añadir valor al producto o ayudar en cualquier aspecto
- **Rojo**: Cada participante puede escribir un par de ideas, acciones o propuestas

Cuestiones de tamaño

¿Qué es un epic?

- Un epic **no** es una historia de usuario grande. Es un **hueco reservado** en la pila de producto para un bloque de trabajo que todavía no se ha desglosado.
- De hecho ni siquiera se escribe en formato de historia: *Gestión de contenidos* nombra un área de funcionalidad, no una acción con un beneficio.
- Los tres tamaños con los que se trabaja:

	horizonte	¿entrega valor?
Epic	varios sprints	sí, pero sin desglosar
Historia de usuario	cabe en un sprint	sí, por sí sola
Tarea	un día o menos	no, es un paso técnico

¿Cuándo se parte un epic?

- El criterio práctico es el tamaño: lo que **no cabe en un sprint** es un epic, y habrá que partirlo antes de comprometerlo.
- Pero no se parte al crearlo, sino **cuando se acerca al tope de la pila**. Es el refinamiento que ya vimos en el *backlog grooming*.
- La razón: desglosar por adelantado toda la pila es volver al análisis completo previo, justo lo que criticábamos en COCOMO. El detalle cuesta trabajo y además **caduca** si cambian las prioridades.
- Esto se traslada a la estimación: un epic lleva una **talla gruesa**, no un número fino. El número fino aparece al partirlo, cuando ya se sabe lo que contiene.
- Regla de pulgar: la pila va **detallada arriba y gruesa abajo**.

Descomposición de un epic

- El epic *Gestión de contenidos* podría partirse en historias como:
  - "Como usuario, quiero crear un nuevo contenido."
  - "Como usuario, quiero ver un listado de los contenidos que he creado."
  - "Como usuario, quiero eliminar un contenido de mi lista."
- Cada trozo debe cumplir **dos** condiciones: caber en un sprint y **entregar valor por sí solo**.
- Hay varias formas de cortar: por **operación** (como arriba), por **paso del flujo**, por **regla de negocio** o por **variante de los datos**.
- Lo que no funciona es cortar por **capa técnica** (primero la base de datos, luego la API, luego la interfaz): incumple la segunda condición, porque ninguna capa sirve de nada por separado.

Epics bien y mal usados

- Para qué sirven de verdad:
  - Mantener la pila **legible**: un epic ocupa una línea donde habría cuarenta historias.
  - Permitir **priorizar** a grandes rasgos sin haber analizado el detalle.
- Dos formas habituales de usarlos mal:
  - El epic que **nunca se parte**: lleva meses en la pila, nadie recuerda qué contiene y su talla ya no significa nada.
  - El epic usado como **categoría**: *Frontend*, *Rendimiento*, *Deuda técnica*. Eso es una etiqueta temática, no un epic: no tiene final y nunca se puede dar por terminado.

Escalando Scrum para Equipos Grandes

¿Por qué escalar Scrum?

- Scrum funciona mejor con equipos pequeños de entre 3 y 9 personas.
- Sin embargo, los grandes proyectos pueden requerir más personas, lo que implica la necesidad de escalar Scrum.
- Escalar Scrum implica coordinar varios equipos para que trabajen en el mismo producto de manera efectiva.

Frameworks para Escalar Scrum

- Existen varios frameworks que ayudan a escalar Scrum, entre los más conocidos están:
  - **SAFe (Scaled Agile Framework)**: Organiza múltiples equipos Scrum en grandes grupos llamados **Agile Release Trains (ARTs)**, que operan sincronizados hacia un objetivo común.
  - **LeSS (Large-Scale Scrum)**: Utiliza un enfoque simple con un único backlog compartido y un Product Owner para todos los equipos.
  - **Nexus**: Coordina la colaboración entre varios equipos Scrum, agregando eventos adicionales para gestionar dependencias.

Ejemplo de Escalado con SAFe

- En SAFe, los equipos Scrum trabajan en grupos más grandes llamados **Agile Release Trains (ARTs)**, que coordinan el trabajo entre múltiples equipos.
- En una gran empresa de desarrollo de software, cada ART puede estar compuesto por 5-12 equipos Scrum que colaboran para lanzar incrementos de software cada 8-12 semanas.
- Estos ARTs trabajan en sincronización para lograr objetivos estratégicos a mayor escala.

Ejemplo de Escalado con LeSS

- **LeSS (Large-Scale Scrum)**: Se enfoca en mantener la simplicidad y utiliza un único backlog compartido y un único Product Owner.
- Todos los equipos trabajan en incrementos pequeños, manteniendo un alto grado de coordinación y colaboración.
- **Ejemplo:** En una startup con varios equipos trabajando en el mismo producto, LeSS permite coordinar de manera sencilla múltiples equipos, sin necesidad de roles o estructuras adicionales.
- Los equipos se sincronizan a través de reuniones conjuntas y planificación compartida.

Ejemplo de Escalado con Nexus

- **Nexus**: Es una extensión de Scrum que permite coordinar entre 3 y 9 equipos Scrum que trabajan juntos en un solo producto.
- `https://www.scrum.org/resources/nexus-guide`
- Nexus introduce un "Nexus Integration Team" que se encarga de la integración continua y la resolución de dependencias entre los equipos.
- **Ejemplo:** Una empresa tecnológica que desarrolla una plataforma de e-commerce podría utilizar Nexus para gestionar la colaboración entre varios equipos que trabajan en módulos diferentes (pago, catálogo de productos, análisis de datos, etc.).

Nexus: Scrum para equipos múltiples

- **Nexus** es un marco ligero para escalar Scrum cuando trabajan entre **3 y 9 equipos** sobre un **único producto**.
- Fue creado por **Ken Schwaber** y está respaldado por *Scrum.org*.
- Su objetivo principal es **minimizar dependencias** y asegurar una **integración continua** del producto.
- Mantiene los principios de Scrum, añadiendo solo lo necesario para coordinar múltiples equipos.

¿Por qué escalar Scrum con Nexus?

- Scrum funciona muy bien con un equipo, pero surgen problemas cuando hay varios:
  - Dependencias técnicas entre equipos.
  - Integraciones tardías que generan conflictos.
  - Trabajo duplicado o mal alineado.
- Nexus aporta **coordinación estructurada** sin añadir burocracia innecesaria.
- Se centra en un concepto clave: **"Integration over coordination"**.

Roles en Nexus

- **Equipos Scrum** son de 3 a 9 equipos, cada uno con PO, SM y Developers.
- **Nexus Integration Team (NIT):**
  - Responsable de asegurar que todos los incrementos se integran correctamente.
  - Puede incluir Product Owner, Scrum Master y desarrolladores con foco en integración.
  - No mandan sobre los equipos, **facilitan y eliminan dependencias**.

Artefactos en Nexus

- **Nexus Product Backlog:** una única lista priorizada para todos los equipos.
- **Refined Nexus Sprint Backlog:**
  - Resultado de una refinación conjunta.
  - Los equipos identifican dependencias y planifican quién hará qué.
- **Integrated Increment:** al final del sprint debe existir **un único entregable integrado**, no varios incrementos independientes.

Eventos en Nexus

- **Nexus Sprint Planning**
  - Reunión coordinada para **alinea objetivos entre equipos**.
  - Se detectan dependencias y se pacta el orden de trabajo.
- **Nexus Daily Scrum**
  - Scrum breve entre representantes de los equipos.
  - Foco en **bloqueos e integración**, no en el trabajo individual.

Eventos en Nexus

- **Sprint Review Nexus:**
  - Se muestra **un único incremento integrado** al cliente.
- **Nexus Retrospective:**
  - Primero reflexión a nivel Nexus (problemas de integración o coordinación).
  - Después, cada equipo realiza su retrospectiva individual.

Beneficios de Nexus

- Favorece la **integración continua** entre equipos.
- Estructura ligera para equipos grandes sin caer en burocracia.
- Evita el "Scrum en silos" (equipos que desarrollan sin comunicación real).
- Mejora la **coherencia del producto final** entregado al cliente.
- Facilita conversaciones técnicas sobre dependencias y arquitectura.

End of Part IV

*Incluye contenido modificado bajo Creative Commons 3.0 de Mike Cohn